PADE: A Predictor-Free Sparse Attention Accelerator via Unified Execution and Stage Fusion

Huizheng Wang†, Hongbin Wang†, Zichuan Wang†, Zhiheng Yue†, Yang Wang†, Chao Li‡, Yang Hu†✉, Shouyi Yin†*
†School of Integrated Circuits, BNRist, Tsinghua University, Beijing, China, 100084
‡School of Computer Science and Engineering, Shanghai Jiao Tong University, Shanghai, China, 200240
*Shanghai Artificial Intelligence Laboratory, Shanghai, China, 200433
✉Corresponding author, hu_yang@tsinghua.edu.cn

2026 IEEE International Symposium on High Performance Computer Architecture (HPCA) | 979-8-3315-9302-5/26/$31.00 ©2026 IEEE | DOI: 10.1109/HPCA68181.2026.11408448

***Abstract*—Attention-based models have revolutionized AI, but the quadratic cost of self-attention incurs severe computational and memory overhead. Sparse attention methods alleviate this by skipping low-relevance token pairs. However, current approaches lack practicality due to the heavy expense of added sparsity predictor, which severely drops their hardware efficiency.**

This paper advances the state-of-the-art (SOTA) by proposing a bit-serial enable stage-fusion (BSF) mechanism, which eliminates the need for a separate predictor. However, it faces key challenges: 1) Inaccurate bit-sliced sparsity speculation leads to incorrect pruning; 2) Hardware under-utilization due to fine-grained and imbalanced bit-level workloads. 3) Tiling difficulty caused by the row-wise dependency in sparsity pruning criteria.

We propose PADE, a predictor-free algorithm-hardware co-design for dynamic sparse attention acceleration. PADE features three key innovations: 1) Bit-wise uncertainty interval-enabled guard filtering (BUI-GF) strategy to accurately identify trivial tokens during each bit round; 2) Bidirectional sparsity-based out-of-order execution (BS-OOE) to improve hardware utilization; 3) Interleaving-based sparsity-tiled attention (ISTA) to reduce both I/O and computational complexity. These techniques, combined with custom accelerator designs, enable practical sparsity acceleration without relying on an added sparsity predictor. Extensive experiments on 22 benchmarks show that PADE achieves $7.43\times$ speed up and $31.1\times$ higher energy efficiency than Nvidia H100 GPU. Compared to SOTA accelerators, PADE achieves $5.1\times$, $4.3\times$ and $3.4\times$ energy saving than Sanger, DOTA and SOFA.

I. Introduction

Transformer models have achieved significant success in various fields, spanning from content generation [109], [5], [48], [76] to computer vision [28], [148], [7]. However, the self-attention mechanism used in Transformers suffers from quadratic time and memory complexity, limiting their scalability to long sequences.

To address this, *sparse attention* techniques have emerged as a promising solution, where attention is computed over a subset of query-key (Q-K) pairs instead of a dense attention matrix. Existing sparse attention works can be divided into two routes: *static sparsity (SS)* and *dynamic sparsity (DS)*. SS [90], [16], [95], [149], [9], [57], [27], [62] relies on predefined sparse patterns that remain fixed during inference, lacking flexibility and often resulting in significant accuracy degradation [73], [35], [59]. In contrast, DS [19], [64], [148], [120], [110], [17], [89], [30], [20], [70], [122], [94], [42], [43], [82], [96], [145], [153], [124], [119], [77], [125], [75], [128] adapts the sparsity pattern at runtime, offering improved accuracy and flexibility, making it more suitable for a broader range of tasks and input types.

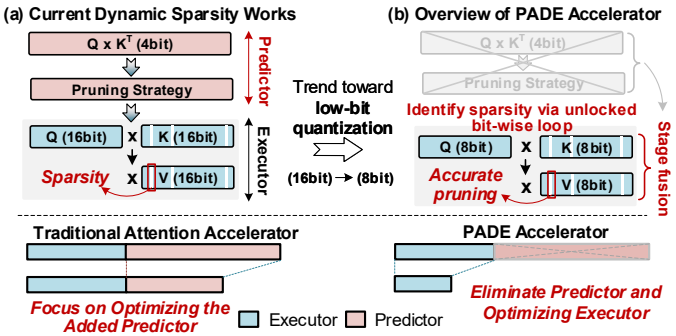


Fig. 1. Comparison of (a) current DS works and (b) PADE.

However, such flexibility of DS comes at the expense of an additional sparsity predictor. Fig. 1 (a) outlines the typical workflow of existing DS attention accelerators [42], [43], [82], [96], [153], [145], [17], [77], [75], [119], [94], [122], which consists of three stages. First, attention scores ($\mathbf{Q}\times\mathbf{K}^T$) are estimated via low-overhead techniques, such as 4-bit MSB multiplication [82], [153], log-domain shifting [94], [122], low-rank approximation [43], [96], and clustering [17], [68]. Next, a pruning strategy, like threshold comparison [82], [75], [153] or top-k sorting [119], [94], [122], generates a sparsity mask for important QK pairs (iQKs). This process relies on an additional sparsity predictor. Finally, only iQKs are processed by the attention executor with higher bit-width precision (typically 16-bit), while ineffective QK-pairs (iEQKs) are directly pruned.

Observation: Unfortunately, modern Transformer models exhibit two prevailing trends: (1) Low-bit quantization, as exemplified by GPTQ [34], LLM.int8() [26], SmoothQuant [140] and Atom [152]; (2) Continuously expanding context length, evolving from GPT-2's 1k context length [97] to Llama-3's 8k-token context [38]. **Under these scenarios, the inclusion of an additional predictor incurs substantial overhead, increasingly offsetting the benefits of sparsity.**

Fig. 2 (a) shows the power breakdown of dense attention and two representative DS accelerators: Sanger [82], SOFA [122], with varying executor bit-widths. Power consumption is categorized into executor and predictor components. Sanger

Authorized licensed use limited to: University of Electronic Science and Tech of China. Downloaded on October 31,2026 at 10:33:52 UTC from IEEE Xplore. Restrictions apply.

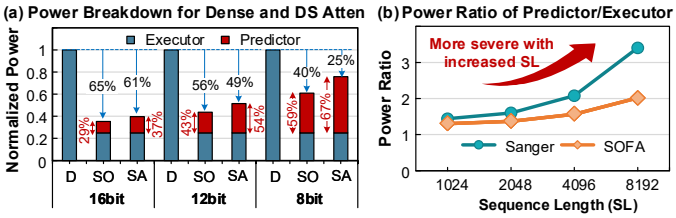


Fig. 2. (a) Power breakdown of dense and DS attention (SA: Sanger, SO: SOFA) with TSMC 28nm across executor bit-widths of Llama7B. (b) Power ratio of predictor and executor versus SL with under 8-bit quantized executor.

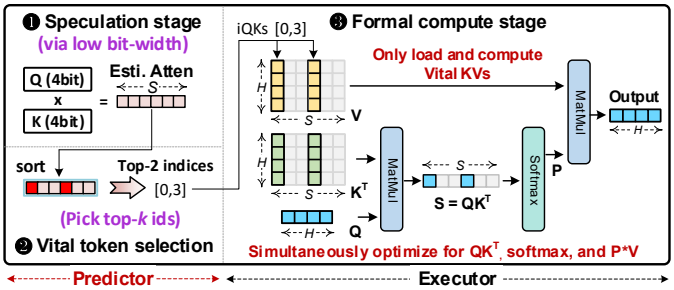


Fig. 3. Illustration of the DS attention mechanism.

uses 4-bit MSB multiplication with threshold comparison, while SOFA employs log domain shifting with top-k sorting. As depicted in Fig. 2(a), there are two key observations: (**1**) At larger executor bit-widths (e.g., 16bit), the DS reduces overall power by about 63%, with the predictor costing only 33%. This explains why previous DS works adopt additional sparsity predictors. (**2**) However, as executor bit-width decreases, predictor overhead becomes dominant. At 8-bit, overall savings drop to merely 32%, with the sparsity predictor occupying over 63% of total cost. This is due to the predictor must access and process full-sized K tensors, a cost unaffected by sparsity.

Further, Fig. 2(b) reveals the predictor-to-executor power ratio under varying sequence lengths (SL). As can be seen, as the SL increases, this ratio grows noticeably for both designs, indicating the growing relative overhead of the predictor. This is because the increased sparsity in longer sequences exacerbates the predictor's relative overhead.

Takeaway: These observations highlight a critical issue: the predictor's overhead increasingly offsets the benefits of sparsity, particularly in the context of modern quantization and longer sequence lengths. **This highlights a need to reduce or even eliminate the prediction overhead.**

Insight: We pinpoint the root cause of the excessive prediction cost stems from the decoupling between existing sparsity predictors and executors, which hinders the computational and memory access efforts paid in the predictor from being reused by the executor. To address this, we draw inspiration from bit-serial computing [58], [1], [67], [15], [60], [53], [40], which separates an INT operation into multi-round bit-level steps. This motivates a unified design that integrates prediction and execution into a single computation stage, thereby eliminating the separate predictor and improving overall efficiency.

To realize this idea, we propose a bit-serial-enable stage fusion (BSF) strategy that eliminates the additional prediction stage via the following key steps: 1) Start with the first bit plane (i.e., MSB) of Keys for bit-serial speculating of $\mathbf{Q}\times\mathbf{K}^T$. 2) Once a token (Key) is runtime identified as unlikely to be an important QK pair (i.e., iQK), its processing and associated memory access with subsequent bit planes are immediately terminated. Otherwise, the next bit plane is fetched on demand for further decision. In this way, the accelerator only needs to perform the remaining computation for the iQK, and obtains the final result by reusing and accumulating the previously generated partial results.

Despite its potential, realizing a BSF-style DS accelerator presents several challenges: **(1)** Lack of an effective bit-wise decision mechanism for early iEQKs identification, due to inaccuracies in bit-grained speculation. **(2)** Hardware under-utilization arises from both cross-bit-plane level PE execution stalls caused by latency variation under fine-grained, on-demand DRAM fetches, and intra–bit-plane level PE load imbalance. **(3)** Tiling difficulty caused by the mismatch between early-termination decisions and row-wise dependency in sparsity pruning criteria.

To this end, we propose PADE, a software-hardware co-design, whose high-level overview is depicted in Fig.1 (b). It features three key techniques that correlate to three challenges:

1) We propose Bit-level Uncertainty Interval-enabled Guarded Filtering (BUI-GF) to accurately identify iEQKs at each bit round. By exploiting properties of two's complement representation, we define a Bit-wise Uncertainty Interval (BUI) to bound the potential variation of inner products. This safety margin enables precise and reliable early pruning decisions.

2) We propose a Bidirectional Sparsity-based Out-of-order Execution (BS-OOE) to sustain high hardware utilization under on-demand memory access. BS-OOE relaxes the cross-bit-plane constraint by allowing PEs to temporarily bypass stalled bit-plane dependencies and execute other ready bit-plane tasks from different tokens, effectively hiding cross-bit-plane memory latency and mitigating execution imbalance. To further improve execution regularity within individual bit planes, we employ bidirectional sparsity as a supporting substrate that reduces intra–bit-plane workload variance and provides a more balanced pool of executable tasks.

3) We propose an Interleaving-based Sparsity-Tiled Attention (ISTA) mechanism to improve IO efficiency. By exploiting the monotonicity of Softmax and the early termination property of bit-serial computation, we skillfully decompose pruning decisions to the tiling level. Additionally, an interleaved update strategy further minimizes redundant operations across tiles.

To support the above optimization mechanisms effectively, we design a dedicated accelerator: **1)** For BUI-GF, it employs the dedicated scoreboard-based, result-reusable PE lane to eliminate redundant memory access across bit execution rounds, thus significantly reducing the energy overhead of repeatedly loading bit planes. **2)** For BS-OOE, PADE integrates scoreboard-based PE lanes to facilitate partial-sum buffering and reuse. **3)** For ISTA, a reuse-aware reorder scheduler is dedicated to improving tiling execution efficiency, by minimizing redundant memory access. The PADE accelerator achieves an

Authorized licensed use limited to: University of Electronic Science and Tech of China. Downloaded on October 31,2026 at 10:33:52 UTC from IEEE Xplore. Restrictions apply.

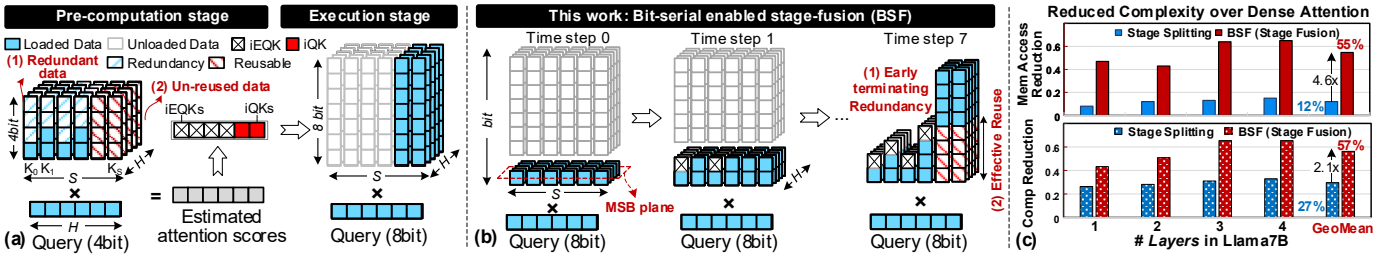


Fig. 4. (a) Traditional DS works, featuring stage splitting. (b) Our work features stage-fusion. (c) Reduced complexity for stage splitting and stage fusion.

average energy efficiency of 11740 GOPS/W, which is 31.1×, 5.1×, 4.3× and 3.4× higher than H100 GPU, SOTA accelerator Sanger, DOTA and SOFA.

II. Background

A. Transformer and DS Attention

Transformer models. Initially, the Transformer maps a length- S sequence into Q, K, and V spaces. Next, Q and K are multiplied to generate an attention score $\mathbf{S}$ with $\mathbb{R}^{S\times S}$, which captures token-to-token correlations. The $\mathbf{S}$ is then passed through a *softmax* and multiplied with V activation, resulting in a matrix $\mathbf{O} \in \mathbb{R}^{S\times H}$, where H denotes hidden dimension. Finally, a feed-forward network (FFN) generates the outputs.

Dynamic Sparsity (DS) attention. Typically, dense attention involves load and compute all Ks and Vs. In contrast, the DS attention fetches and processes only the important iQK pairs. As depicted in Fig. 3, only the 0th and 3rd KVs are loaded and computed with the Query, thus effectively reducing the computation and memory access. However, this reduced complexity comes at the cost of an additional sparsity predictor, which involves low-bit QK speculation and vital token selection processes. This introduces non-negligible overhead.

III. Motivation

A. Re-examinating DS Works and Opportunity

As shown in Fig. 4(a), traditional DS works adopt a *stage-splitting* paradigm that decouples prediction from execution. Taking Sanger [82] as an example, the predictor takes the full 4-bit Key tensor to identify iQKs, while the executor separately fetches the corresponding KVs for precise computation. **However, this paradigm introduces two inefficiency sources**: (**1**) For Keys related to iEQKs (e.g., K_0), a single bit may suffice to identify their insignificance. However, the stage-splitting approach blindly loads 4 bits, resulting in redundancy. (**2**) For Keys related to iQKs (e.g., K_s), the executor reloads their high-bit-width versions for subsequent more precise computation. However, it fails to reuse the data already processed during prediction, leading to inefficiency.

Quantifying predictor overhead. As shown in Fig. 2, after sparsification, the added predictor incurs over 63% power overhead, limiting the gain of DS attention to just 1.5× over dense attention. To this end, we derive the design guidance:

> **(Design Guidance)** An ideal DS accelerator should eliminate the extra sparsity predictor while preserving sparsity.

Motivated by this insight, the BSF strategy aims to unify prediction and execution within a single computation stage, thereby minimizing both computation and memory access. As illustrated in Fig. 4(b), BSF progressively, on-demand extracts lower-order bit-planes to assess token importance during QK computation. This enables not only early termination for iEQKs but also computation and memory access reuse for iQKs.

Unlike stage-splitting DS designs that rely on separate predictors, BSF eliminates prediction overhead and enables fine-grained early termination to reduce unnecessary computation and memory access. Fig. 4(c) profiles the memory access and computation reduction achieved by different strategies in the attention modules across four layers from LLaMA-2 7B [116]. On average, BSF can achieve 4.6× higher memory access and 2.1× more computation reduction, compared to traditional stage-splitting DS approaches, highlighting its significant energy efficiency advantage over current DS methods.

B. Challenges for Bit-serial Enable Stage-fusion

Despite its theoretical benefits, a naive implementation of BSF will encounter three challenges, as depicted in Fig. 5.

> **(Challenge 1)** Incorrect pruning decisions caused by the inherent inaccuracy in fine-grained bit-wise speculation.

As shown in the Fig. 5 (a), the 1-bit MSB representation of (+5) and 1-bit representation of (-5) are used to predict the result of (+5) × (+5) + (+5) × (-5). The true result should be 0. However, under the 1-bit representation, the MSB bit plane of $(1011)_2$ (-5) and $(0101)_2$ (+5) are regarded as $(1000)_2$ (-8) and $(0000)_2$ (+0), respectively. This leads to an estimated result of -40, which significantly deviates from the correct result. This severe error can lead to incorrect token pruning during early termination. For example, in K_7 of Fig. 5 (b).

Key idea. Inspired by the conservative margin concept [75], [89], we propose bit uncertainty interval-enabled guarded filtering (BUI-GF), a lightweight mechanism for simple yet accurate token pruning. BUI-GF characterizes the maximum possible fluctuation of dot products across bit planes using simple yet efficient bit flipping, enabling conservative yet hardware-efficient pruning decisions with minimal overhead. To support this, we design a scoreboard-based result-reuse processing element (PE) to reduce the hardware overhead of BUI and minimize redundant bit-plane memory accesses.

Authorized licensed use limited to: University of Electronic Science and Tech of China. Downloaded on October 31,2026 at 10:33:52 UTC from IEEE Xplore. Restrictions apply.

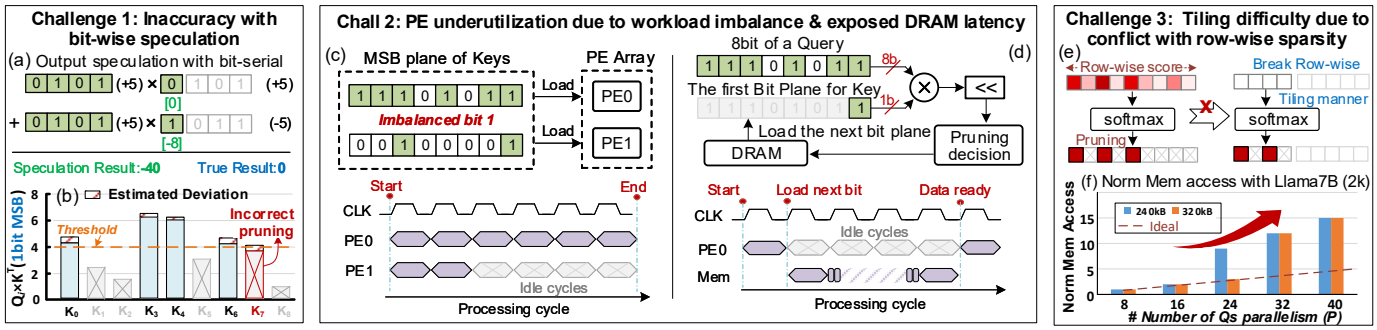


Fig. 5. Challenges for bit-serial enable stage fusion. (a)-(b) Inaccuracy (c)-(d) Hardware under-utilization. (e)-(f) Tiling difficulty.

TABLE I
SUMMARY FOR SOTA ATTENTION ACCELERATORS.

Accelerator	Optimization		Predictor Free	Tiling Support	Optimiz. Level
	Computation	Memory			
ELSA [43]	✓	×	×	×	Value
Sanger [82]	✓	×	×	×	Value
DOTA [96]	✓	×	×	×	Value
DTATrans[145]	✓	Low	✓*	×	Value
SpAtten [119]	✓	Low	✓*	×	Multi-bit
Energon [153]	✓	×	×	×	Multi-bit
FACT [94]	✓	×	×	×	Value
SOFA [122]	✓	Low	×	✓	Value
PADE	✔	✔	✔	✔	**Bit**

* Sparsity guided by preceding layer scores; Accuracy degradation w/o retrain.

(Challenge 2) Compute resource under-utilization caused by cross/intra-bit-plane execution stalls.

In BSF, bit-plane–granular, on-demand execution tightly couples computation and memory access to runtime QK decisions, inherently amplifying both and cross–bit-plane DRAM latency exposure and intra–bit-plane workload imbalance, which together lead to severe PE underutilization.

At cross-bit-plane level, as shown in Fig. 5 (d), to enable runtime, bit-grained early termination, it is critical to avoid the bulk loading of all bit-planes for each Key. Instead, each Key should be evaluated bit-plane by bit-plane to determine whether it qualifies as an iEQK. If true, the subsequent bit-planes for that Key are skipped; otherwise, the next bit-plane is required. However, due to DRAM's dynamic precharge mechanism, the diverse bit-planes are fetched on demand at fine granularity, typically incurring several dozen cycles of latency [12], [56]. Naively stalling computation during data loading results in underutilized computational resources.

At intra-bit-plane level, as depicted in Fig. 5 (c), each bit-plane of different Keys is assigned to a separate PE for parallel execution. However, PEs handling bit planes with more '1' bits incur longer execution cycles (e.g., PE 0), while those with fewer '1' bits complete earlier (e.g., PE 1), resulting in computation stalls and PE underutilization.

Key idea. This observation indicates that system-level performance must be treated as a high-dimensional scheduling problem that considers cross-bit-plane dependencies and DRAM access latency, which lies beyond the scope of existing intra–bit-plane–centric optimizations such as [41], [15], [113]. To address this challenge, we propose Out-of-Order Execution (OOE) as the primary mechanism for mitigating cross-bit-plane latency. The core idea is to decouple the execution order of bit planes from their logical significance order, allowing a PE to make forward progress on other ready bit-plane tasks while waiting for delayed memory responses. On top of this general OOE framework, we adopt bidirectional sparsity (BS) from [15] as an effective supporting substrate to further regularize execution. By integrating OOE with BS, the BS-OOE achieves both (1) robustness to cross-bit-plane DRAM latency variation and (2) improved load balance within individual execution rounds.

(Challenge 3) IO inefficiency resulting from the conflict between tiling with the row-dependent pruning strategy.

As shown in Fig. 5 (e), existing pruning strategies rely on row-wise attention score distributions to assess token importance, introducing strong row dependencies. This dependency prevents effective tiling, which is essential for IO efficiency. Without tiling, when the number of parallel queries increases, memory access overhead grows sharply. As illustrated in Fig. 5 (f), increasing the number of parallel queries (P) from 8 to 32, leads to over 12× more memory accesses. A coarse solution is to enlarge on-chip SRAM, but this incurs significant area inefficiency. For example, with (P=512, S=2048), 5MB of SRAM is required, resulting in a 5.47 mm^2 footprint under TSMC 28nm technology, which is 7.4× and 8.9× larger than the total area of SpAtten [119] and ELSA [43], respectively.

Key idea. We reveal and leverage the monotonicity of the softmax function, and further adjust the pruning decision mechanism as follows: retaining tokens that reach the least significant bit (LSB) plane but without being pruned. This enables efficient and I/O-friendly pruning within tiled regions.

Unfortunately, current attention accelerators still suffer from computation and memory access inefficiencies, as they fail to exploit bit-grained opportunities to eliminate the high-overhead predictor. Table I summarizes their features. The majority of existing works [42], [43], [82], [96], [145] focus on accelerating attention by alleviating computation overhead. For example, ELSA [43], Sanger [82], DOTA [96], FACT [94] adopt techniques like binary hashing, half-precision MSB, low-rank approximation, log-domain shifting to accelerate computation. However, these methods overlook memory optimization. While SpAtten [119]. SOFA [122] realizes this challenge, their coarse-grained strategies, like hybrid quantization, and cross-stage tiling, fail to exploit fine-

Authorized licensed use limited to: University of Electronic Science and Tech of China. Downloaded on October 31,2026 at 10:33:52 UTC from IEEE Xplore. Restrictions apply.

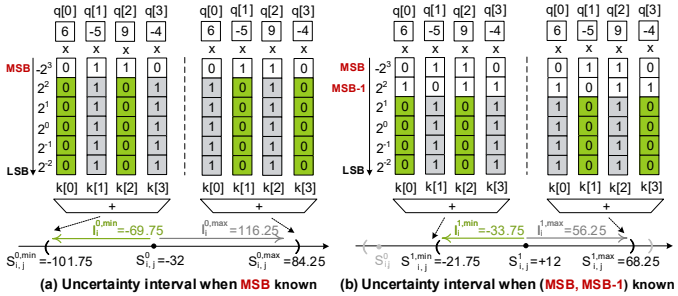


Fig. 6. Illustration of BUI with dot-product $\mathbf{Q}_i \times \mathbf{K}_j$.

grained, bit-level optimizations. Further, all current works rely on extra sparsity predictors, incurring substantial overhead. Notably, DTATrans [145] and SpAtten [119] guide sparsity using attention scores from the previous layer. While this strategy partially reduces predictor overhead, it necessitates resource-intensive retrain to recover accuracy. **These limitations motivate us to design an efficient attention accelerator that jointly optimizes computation and memory at fine granularity, while eliminating the sparsity predictor.**

IV. Algorithm Optimizations of PADE

To effectively support the BSF strategy, we propose three key optimizations: BUI-GF, BS-OOE, and ISTA. BUI-GF ensures precise pruning in bit-wise operations, BS-OOE optimizes hardware utilization, and ISTA maintains sparsity while enhancing I/O efficiency via tiling attention.

A. BUI-enabled Guarded Filtering (BUI-GF)

We begin by analyzing the mathematical properties of the softmax function, revealing its intrinsic potential for a simple yet efficient max-based pruning decision. Without loss of generality, consider a two-element input vector $[x_0, x_1]$, where x_1 is the max element and $x_1{=}x_0{+}\Delta$. As depicted in Eq.(1), the softmax output for x_0 decays exponentially with the gap Δ from x_1. This indicates that an element's contribution diminishes rapidly as its distance Δ from the max increases.

$$x_1{=}x_0{+}\Delta \Rightarrow \mathrm{softmax}(x_0){=}\frac{e^{x_0}}{e^{x_0}{+}e^{x_0+\Delta}}{=}\frac{1}{1{+}e^{\Delta}} < \frac{1}{e^{\Delta}} \quad (1)$$

Naturally, a straightforward way to combine BSF with a max-based pruning strategy is to estimate the attention via partial bit planes of Keys, identify the max value, then use it to define a pruning threshold. However, such a crude method will incur severe estimation error, as analyzed in §III-B.

To this end, we propose a *bit-level uncertainty interval (BUI)* enabled guarded filtering (BUI-GF). We first introduce the BUI, which quantifies the potential variation in a dot-product, i.e., $\mathbf{Q}_i\mathbf{K}_j$ caused by the remaining bit planes of $\mathbf{K}_j$. The BUI extends the margin concept [89], [75] for bit-grained 2's complement number format. Specifically, for a p-bit integer $b_{p-1}b_{p-2}...b_0$ with 2's complement format, its value x is:

$$x = -b_{p-1}2^{p-1} + \sum_{i=0}^{p-2} b_i 2^i. \quad (2)$$

In this format, all bits except the sign bit (b_{p-1}) contribute a non-negative value, meaning that each additional bit can only

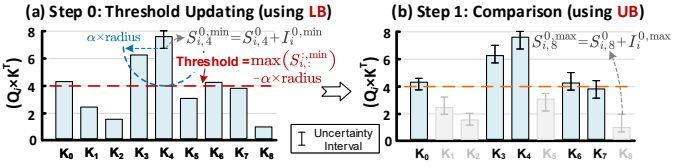


Fig. 7. Illustration of the BUI-GF strategy.

increase or maintain the magnitude. Based on this property, Figs. 6 (a)(b) exemplify the BUI. In the example, $\mathbf{Q}_i$ denotes the i-th row of $\mathbf{Q}$ matrix, containing four entries with full 8-bit precision, while $\mathbf{K}_j$ is bit-serially processed: 1 bit in Fig. 6 (a) and 2 bits in Fig. 6 (b). For positive elements of $\mathbf{Q}_i$, BUI sets the unknown bits of $\mathbf{K}_j$ to 1 (shown in blue) and for negative entries, it set them to 0 (shown in red), yielding the potential largest score $S_{i,j}^{r,\max}$, as they account only for positive contributions. Here, $r{\in}[0,7]$ denotes the number of processed bit planes in 8-bit quantization. Conversely, BUI flips the unknown bits to obtain the potentially smallest score $S_{i,j}^{r,\min}$. We model this process as follows:

$$S_{i,j}^{r,\min} = S_{i,j}^{r} + I_i^{r,\min}, \quad S_{i,j}^{r,\max} = S_{i,j}^{r} + I_i^{r,\max}, \quad (3)$$

where $S_{i,j}^{r}$ denotes the conservative value by setting all unknown bits of $\mathbf{K}_j$ to zero, and $I_i^{r,\min}$ and $I_i^{r,\min}$ are the uncertainty intervals decided only by $\mathbf{Q}_i$. For example, in Fig. 6 (a), we only know the MSB of $\mathbf{K}_j$, and by applying Eq. (3), the BUI is determined to be the low bound (LB) $S_{i,j}^{0,\min} = -101.75$ and the upper bound (UB) $S_{i,j}^{0,\max} = 84.25$.

Based on the BUI, we propose the BUI-GF strategy, which consists of two main steps, as illustrated in Fig. 7. First, the BUI-GF determines the pruning threshold using the Eq. (4):

$$\mathcal{T} = \max(S_{i,:}^{:,\min}) - \alpha \times radius, \quad 0 \le \alpha \le 1, \quad (4)$$

where $S_{i,:}^{:,\min}$ denotes the LBs of the estimated attention score for the i-th row. Notably, the scores are not confined to a specific bit plane, but are derived from all current processed bit planes. Based on our experiments, we set the default *radius* to 5 and use a parameter $\alpha{\in}[0,1]$ to control the threshold $\mathcal{T}$. By adjusting the α, we can control the pruning ratio (DSE see §VI-D). Then, the BUI-GF compares the UBs of attention scores with the threshold, retaining the Ks (e.g. 0,3,4,6,7), whose attention values are greater than this threshold.

B. BS-OOE: Improving Hardware Utilization

The BUI-GF strategy enables accurate pruning decisions based on partial scores, which are incrementally estimated using bit-wise planes of the Key tensor. As analyzed in §III-B, improving hardware utilization in bit-serial attention is not merely an intra–bit-plane workload balancing problem, but rather a cross-bit-plane execution scheduling problem under system-level memory latency uncertainty.

To address these issues, we propose BS-OOE, which integrates out-of-order execution with bidirectional sparsity. OOE serves as the core execution mechanism at the cross-bit-plane level, sustaining high hardware utilization under dynamic execution conditions by tolerating variable DRAM access latency.

Authorized licensed use limited to: University of Electronic Science and Tech of China. Downloaded on October 31,2026 at 10:33:52 UTC from IEEE Xplore. Restrictions apply.

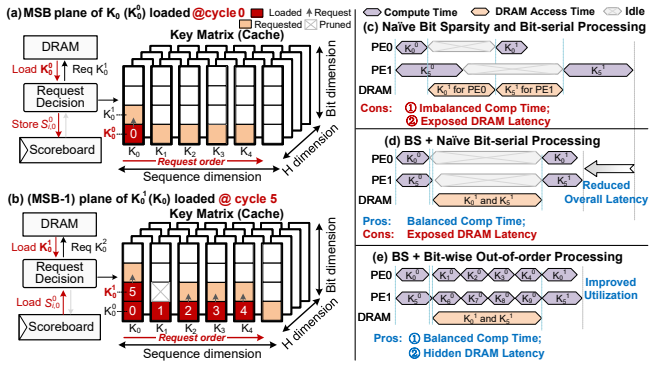


Fig. 8. (a)(b) Bit-wise OOE execution, (c)-(e) benefits of BS-OOE strategy.

At the intra-bit-plane level, bidirectional sparsity complements OOE by mitigating load imbalance across PEs, thereby improving computational regularity within each bit plane.

Cross-Bit-Plane Level: The typical workflow of OOE strategy is illustrated in Fig. 8 (a)(b). ❶ At the beginning of score speculation, only the MSB plane of each Key is fetched. ❷ Whenever a bit plane arrives from DRAM, the corresponding partial score is immediately computed and evaluated by BUI-GF. ❸ If a Key remains unpruned, the next bit plane of that same Key (e.g. K_0^1 in Fig. 8 (a)) is requested, while the current partial score ($S_{i,0}^0$) is checkpointed in a Scoreboard. If the Key is pruned, execution instead advances to the next Key by requesting its first bit plane. Crucially, while waiting for the requested downstream bit plane to be delivered, the PE does not stall. Instead, it continues processing other ready bit-plane tasks from different Keys, such as K_1^0,..,K_4^0. ❹ Once the requested bit plane (e.g., K_0^1 in Fig. 8 (b)) becomes available, the PE restores the associated partial score (i.e., $S_{i,0}^0$) from the Scoreboard and resumes accumulation. In this way, the compute units remain active opportunistically, thereby improving hardware utilization for bit-wise speculation. Notably, OOE is orthogonal to the specific form of bit-plane sparsity.

Intra-Bit-Plane Level: At the intra-bit-plane level, the problem of exploiting bit-level sparsity has been widely explored in prior studies [25], [41], [15], [113]. We build upon a representative bit-plane sparsity approach [15] and adapt it from static weight settings to dynamic attention workloads. In this scenario, a lightweight sparsity-aware scheduler (see §V-D) is indispensable, as sparsity patterns are determined at runtime and vary across the on-demand fetched bit-planes of Keys. Specifically, the core idea of bidirectional sparsity is that bit value '1' is a form of sparsity. Without loss of generality, the dot-product between an N-element Query and Key can be formulated as:

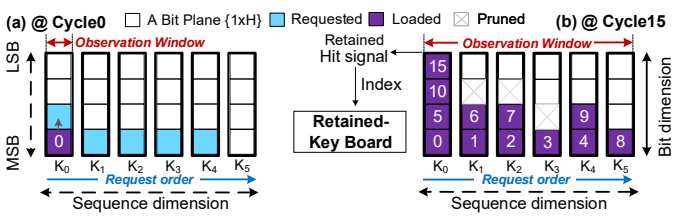


Fig. 9. Tile-level sparsity decision.

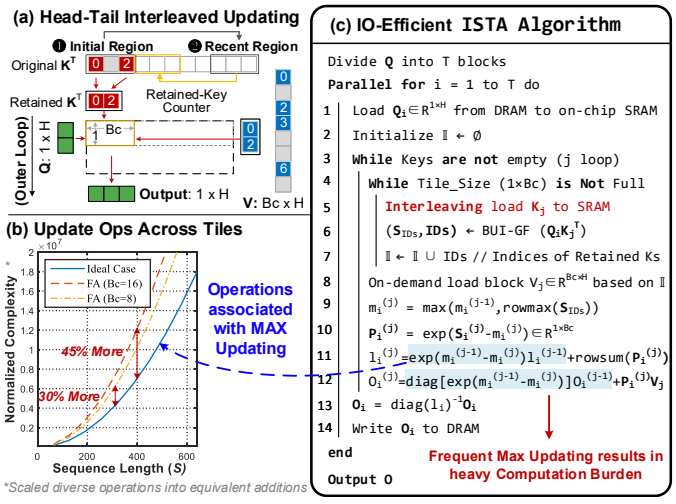


Fig. 10. (a) Illustration for head-tail interleaving updating. (b) Heavy updating cost across tiles. (c) Detailed procedure of the ISTA algorithm.

$$\sum_{j=0}^{N-1} q_j k_j = \sum_{b=0}^{p-1} 2^b \times \sum_{j=0}^{N-1} q_j \times k_j^b, \tag{5}$$

where k_j^b is the b-th bit of element k_j. Since each bit of k_j can only be either 0 or 1, the second partial sum on the right-hand side of Eq. (5) can be reorganized as:

$$\sum_{j=0}^{N-1} q_j \times k_j^b = \sum_{\forall j:k_j^b=1} q_j = \sum_{j=0}^{N-1} q_j - \sum_{\forall j:k_j^b=0} q_j. \tag{6}$$

From Eqs. (5) (6), we observe that instead of accumulating the Query entries corresponding to bit-1, one can equivalently subtract the Query entries corresponding to bit-0 from the total sum of all Query entries. This transformation limits the number of bits involved in computation to at most half, leading to improved load balance across PEs.

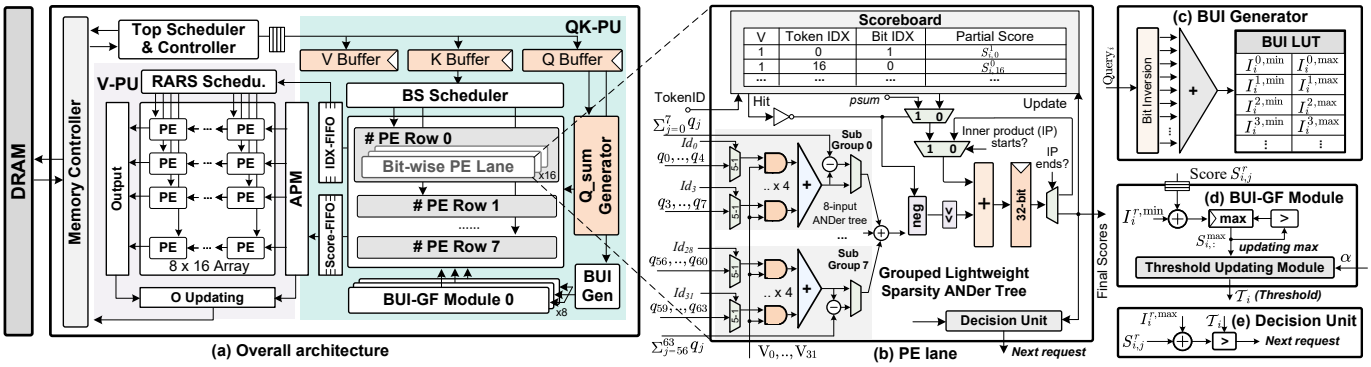
Figs. 8 (c)-(e) highlight the advantages of BS-OOE over naive bit sparsity + bit serial processing, by comparing the timeline of PE0 in the scenarios depicted in Figs. 8 (a)(b). As shown in Fig. 8 (c), naive bit sparsity processing suffers from workload imbalance, causing severe computation time discrepancies across PEs. This results in scattered DRAM accesses, idle cycles, and high latency. In Fig. 8 (d), BS alleviates this imbalance between PEs, enabling memory request merging and reducing row activation during DRAM access. However, computation resources remain underutilized during memory access. In contrast, as depicted in Fig. 8 (e), BS-OOE further leverages bit-wise out-of-order execution to improve resource utilization. Specifically, while PE0 waits for DRAM to return K_0^1, it continues to process $K_1^0, .., K_4^0$, without idling.

C. ISTA: Enhancing IO Efficiency

Reducing unnecessary data movement is critical for supporting ultra-long sequences. While tiling (e.g., FlashAttention) offers a promising solution, it is incompatible with sparse attention because it breaks the row-wise dependency of softmax, on which BUI-GF (§IV-A) relies for max-based pruning.

To address this, we re-examine softmax properties and show that pruning decisions can be performed on subsets with proper

Authorized licensed use limited to: University of Electronic Science and Tech of China. Downloaded on October 31,2026 at 10:33:52 UTC from IEEE Xplore. Restrictions apply.



(a) Overall architecture (b) PE lane

Fig. 11. (a) Overview of the PADE architecture. (b) Bit-wise PE lane. (c) BUI Generator. (d) BUI-GF Module. (e) Decision Unit.

adjustments. Eq. (7) shows that the softmax denominator grows monotonically as more elements are added, due to the non-negativity of exponentials. Therefore, if a token's score falls below the threshold within a subset, its global row-wise softmax score can only be lower. This enables the safe application of the BUI-GF strategy within tiled regions.

$$softmax(x_i) = \frac{\exp(x_i)}{\sum_{j=0}^{N-1}\exp(x_j)} \leq \frac{\exp(x_i)}{\sum_{j\in subset}\exp(x_j)} \quad (7)$$

To identify retained keys within a subset, a key is retained if all its bit planes are processed and it remains unpruned. As depicted in Fig. 9, at cycle 0, only the partial score of K_0 is calculated, with the subset observation window size initialized to 1. By cycle 15, scores relative to six Keys have been computed, expanding the window to size 6. During each bit plane calculation, the BUI-GF works continuously to check if pruning. If a Key (i.e., K_0) remains unpruned after processing its least significant bit (LSB) plane, it is deemed an important token, and its index is stored in the Retained Key Board.

Driven by the tile-level sparsity decisions, the ISTA algorithm is detailed in Fig. 10 (c). Its process begins by performing sparse QK computations, retaining and storing the indices (IDs) and scores (S_{IDs}) of selected Keys (lines 4-6). Once the number of retained entries reaches the tile size B_c (line 3), the corresponding Vs are fetched for subsequent computation (line 8). As more tiles are processed, the maximum value (line 9) and the exponential sum (line 11) are progressively updated and propagated across tiles. Finally, the attention output is computed based on the accumulated results (line 13).

However, a key observation is that naive left-to-right computation causes frequent updates of the maximum across tiles (line 9 in Fig. 10 (c)), leading to a series of redundant operations (line 11). Specifically, each time the maximum is updated, it triggers one subtraction, one exponentiation and two multiplications of scalar and vector (see highlights in lines 11-12). We employ the arithmetic complexity model [10], [123], [121] to normalize the complexity for different operations. As profiled in Fig. 10 (b), for S=2048 and B_c=16, this leads to about 30% increase in computation complexity compared to the vanilla implementation. Also, the overhead becomes more as B_c decreases, due to more frequent updates.

To minimize the overhead associated with the 'max updating', it is advantageous to prioritize dominant tokens. Note that the reduction in operations refers to the redundant computations shown in lines 11–12, rather than the comparison operations in line 9. To this end, we propose a *head-tail interleaved updating* strategy that exploits the locality property of attention: Recently generated tokens and the initial token typically exhibit higher weights than others [115], [57]. As illustrated in Fig. 10(a), the update begins with the initial region, then jumps to the recent region, and subsequently returns to the post-initial region, repeating this interleaved pattern across tiles. This pattern reduces unnecessary maximum value updates, resulting in a 20%-40% reduction in total operations. It is important to note that, without attention locality, the performance of head-tail interleaving is on par with regular execution and not worse.

V. Architecture and Hardware Innovation

A. Architecture Overview

Fig. 11 (a) depicts the overall architecture of the PADE, which incorporates two major components:

1) Query-Key Processing Unit (QK-PU): This unit computes the dot-product of the Query and Key matrices (i.e., $\mathbf{QK}^T$). It includes a PE array, eight BUI-GF modules, a Q_sum generator, a sparsity scheduler, and a BUI generator. The PE array comprises eight rows, each containing 16 bit-wise PE lanes, dedicated to processing a single query. Together, these components support both the BUI-GF and BS-OOE strategies.

2) Value Processing Unit (V-PU): To support ISTA, the VPU unit computes final results from retained (non-pruned) scores in a tiled manner. It comprises an auxiliary processing module (APM) for exponentiation, followed by an 8×16 output-stationary systolic array. To reduce memory access overhead, a RARS scheduler is integrated to enhance data reuse.

B. Overall Dataflow

In PADE, self-attention operands use 8-bit precision, with each Key vector divided into eight 1-bit planes. During the pre-fill stage, PADE processes eight queries within a head, whereas during the decoding stage, it handles different queries across multiple heads. Its detailed process is as follows (Fig. 11 (a)):

First (❶), before the $\mathbf{Q}_i\mathbf{K}^T$ computation, the BUI Generator initializes eight uncertainty interval pairs $(I_i^{r,\min}, I_i^{r,\max})$,

Authorized licensed use limited to: University of Electronic Science and Tech of China. Downloaded on October 31,2026 at 10:33:52 UTC from IEEE Xplore. Restrictions apply.

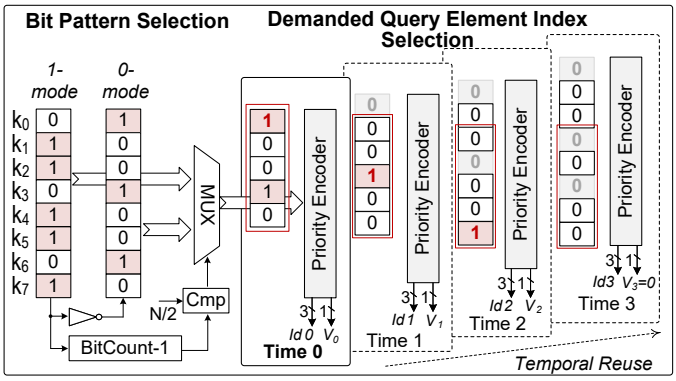


Fig. 12. Temporal reuse architecture of bidirectional sparsity (BS) scheduler.

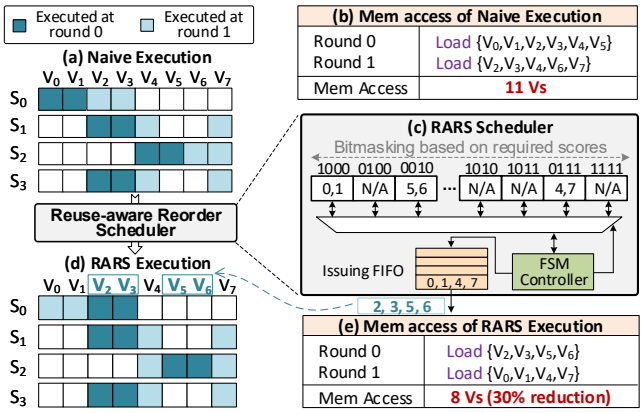


Fig. 13. Reuse-aware reorder scheduler (RARS).

where $r \in [0, 7]$, based on the input $\mathbf{Q}_i$. Each pair corresponds to a specific bit plane. These uncertainty interval pairs are then stored in a lookup table (LUT), as shown in Fig. 11 (c). Next (❷), 16 PE lanes in a PE row perform the dot product for $\mathbf{Q}_i\mathbf{K}^T$ and in an out-of-order manner in parallel. Following this (❸), the BUI-GF Module, as depicted in Fig. 11 (d), calculates the pruning threshold $\mathcal{T}_i$ using the max value among the all current score $S_i^{:,\min}$ with the BUI-GF logic (Eq. (4)). Finally (❹), the pruning threshold $\mathcal{T}_i$ is broadcast to all PE lanes in a row, enabling the evaluation of whether the score of the token j satisfying $S_{i,j}^{r,\max} > \mathcal{T}_i$, as depicted in Fig. 11 (e). If true, the PE lane requests the next bit plane for further computation. Otherwise, the token j is immediately pruned. This process is repeated until the LSB is reached. The remaining scores are sent to V-PU to produce final outputs.

C. ScoreBoard-Based Result Reusable PE Lane

To perform bit-serial speculation for $\mathbf{QK}$, a straightforward solution is to incrementally compute over the bit planes of $\mathbf{K}$. Specifically, it accesses one bit plane (MSB) in the first round, two bit planes (MSB, MSB-1) in the second round, and so on, until the final round. However, repeated memory accesses in this scheme lead to significant power consumption.

To enable result reuse, we dedicate a scoreboard-assisted bit-wise PE lane, as shown in Fig. 11 (b), which comprises three key components: a grouped lightweight sparsity ANDer tree (GSAT) for 64-input dot-product and two modules supporting BUI-GF for out-of-order execution. 1) To avoid repeatedly loading bit planes, each PE lane integrates a scoreboard that temporarily caches partial scores $S_{i,j}^r$ for unpruned tokens. 2) The Decision Unit performs BUI-GF logic, makes pruning decisions and selects the next bit plane to fetch.

These modules operate collaboratively to enable efficient early pruning while minimizing redundant memory access. Initially, the GSAT computes the partial dot product $\Delta S_{i,j}^r$ from an 8-bit vector $\mathbf{Q}_i$ and a 1-bit Key plane $\mathbf{K}_j^r$. Meanwhile, the scoreboard is accessed via the Key's index j. If a previous partial score $S_{i,j}^{r-1}$ exists, it is fetched and updated as $S_{i,j}^r = S_{i,j}^{r-1} + \Delta S_{i,j}^r$. Otherwise, indicated that the current bit plane is MSB, $\Delta S_{i,j}^r$ is directly written to the scoreboard, and Hit signal is pulled down to show no prior score is available.

Decision Unit. The unit determines pruning by receiving the max uncertainty interval $I_i^{r,\max}$ and the partial score $S_{i,j}^r$. It checks whether $S_{i,j}^r + I_i^{r,\max} > \mathcal{T}_i$ holds. If true, it requests the next bit plane of $\mathbf{K}_j$, i.e., $\mathbf{K}_j^{r+1}$, and updates the partial score in the scoreboard. Otherwise, it evicts the token entry from the scoreboard and requests the next Key vector from DRAM.

BUI-GF Module. As shown in Fig. 11 (d), the BUI-GF module reads scores from registers and adds the min uncertainty interval $I_i^{r,\min}$ to compute the lower bound of scores. Based on these values and a predefined ratio α, the Threshold Updating Module applies BUI-GF logic (Eq. (4)) to generate threshold $\mathcal{T}_i$, which is then broadcast to all PE lanes in row$_i$.

D. Grouped Lightweight Sparsity ANDer Tree (GSAT)

To materialize the OOE execution model in hardware, we adopt the bit-serial ANDer tree from [15] and extend it to support 64-dimensional dot products. A straightforward approach to exploiting bit sparsity directly selects and accumulates nonzero activations, but requires large multiplexers. For a 64-dimensional dot product, this coarse design would need at least 32 64-input multiplexers, significantly reducing efficiency.

The overhead reduction is achieved through a two-level decomposition. A 64-dimensional dot product is first decomposed into sub-group–level partial sums, and the structural sparsity induced by BS-OOE further reduces multiplexing overhead. Based on this, we design a grouped lightweight ANDer tree architecture. As shown in Fig. 11(b), the dot product is partitioned into eight 8-dimensional sub-groups. In the worst case, each sub-group selects up to five query elements, implemented using four 5-to-1 multiplexers over sliding windows. Compared to the naive design, MUX cost is reduced (32×64:1 → 32×5:1), with a trade-off of more subtractors and Q_sum generators.

Notably, scoreboard capacity and GSAT design jointly determine the area overhead of each PE lane. While enlarging the scoreboard improves utilization by buffering more partial sums, it also increases area cost; under a fixed PE area budget, this overhead directly reduces the area available for GSAT. Meanwhile, within GSAT, smaller sub-group sizes enhance out-of-order execution scheduling flexibility, whereas larger sub-group sizes increase the likelihood of scoreboard blocking.

Authorized licensed use limited to: University of Electronic Science and Tech of China. Downloaded on October 31,2026 at 10:33:52 UTC from IEEE Xplore. Restrictions apply.

TABLE II

ACCURACY OF DIFFERENT TRANSFORMER MODELS WITH MXINT8, FP16, INT8 AND PADE CONFIGURATIONS (S: STANDARD, A: AGGRESSIVE).

Model	LlaMa2-7B						LlaMa3-8B						OPT1B3	Bloom1B7		Qwen7B		ViT-L/16		PVT	
Task‡	Dolly	Wikili.	MBPP	Wiki2	MMLU	Winog.	Dolly	Wikili.	MBPP	Wiki2	MMLU	Winog.	Wikili.	MBPP	Wikili.	MBPP	Wikili.	MBPP	Image	VTAB	Image	VTAB
MXINT8	36.5	39.3	17.5%	5.63	35.2%	69.8%	40.9	43.6	23.3%	5.01	42.2%	75.1%	36.1	11.9%	44.6	16.3%	46.8	30.5%	85.5%	72.8%	89.7%	77.5%
FP16	36.4	39.1	17.5%	5.71	35.1%	69.4%	40.8	42.7	21.8%	5.11	41.2%	74.2%	36.2	11.9%	44.3	16%	46.6	30%	85.3%	72.7%	89.4%	77.3%
INT8	36.4	38.9	17.2%	5.73	34.7%	69.3%	40.7	42.7	21.6%	5.13	40.9%	73.7%	35.9	11.6%	44.1	15.7%	46.4	29.2%	85.3%	72.5%	89.3%	77.1%
PADE (S)	36.3	38.9	17.2%	5.75	34.6%	69.2%	40.6	42.6	21.5%	5.13	40.7%	73.7%	35.9	11.5%	44.0	15.6%	46.3	29.2%	85.3%	72.5%	89.3%	77.1%
PADE (A)	36.1	38.4	16.5%	5.80	34.1%	68.7%	40.5	42.0	21.0%	5.19	40.2%	72.8%	35.3	11.0%	43.6	15.2%	45.9	28.4%	84.9%	72.4%	89.1%	76.8%

‡ MMLU, WinoGrande, MBPP, Imagenet, VTAB are evaluated by accuracy. Dolly, Wikilingua are evaluated by ROUGE-1. Wikitext2 is evaluated by PPL, where lower is better.

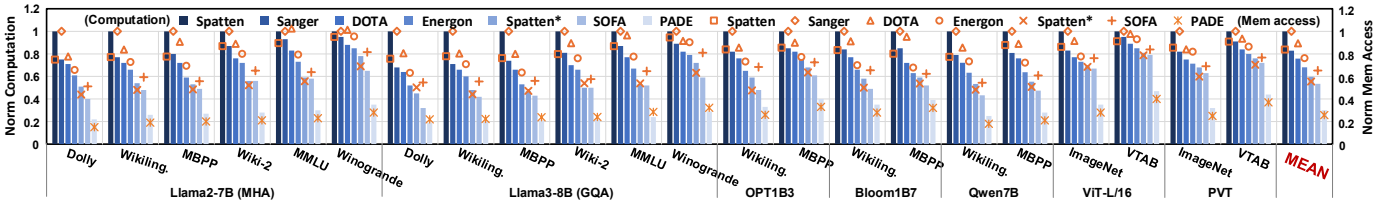


Fig. 14. Normalized computation and memory access across diverse Transformer models and tasks. Spatten* performs additional fine-tuning.

TABLE III

ON-CHIP HARDWARE AND OFF-CHIP DRAM CONFIGURATIONS OF PADE

On-chip Buffer	320KB SRAM for Key and Value buffers; 32KB Q buffer
QK-PU	128 Bit-wise PE Lanes; A Q_sum Generator; 8 BUI-GF modules; A BUI Generator; A Sparsity Scheduler
- Bit-wise PE Lane	64-dim × 8-bit × 1-bit Grouped Sparsity ANDer tree; 32 entry × 45-bit Scoreboard
V-PU	A Systolic Array with 8 × 16 INT8 PEs; A 128-input FP16 APM; A RARS Scheduler
Off-chip DRAM	HBM2; 16×64-bit pseudo channels@2Gbps; Each channel provides 16GB/s; BL=4×64b; tRC=50ns

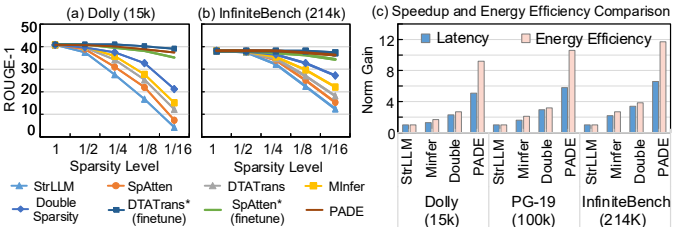


Fig. 15. (a)(b) Accuracy comparison with current sparse attention methods. (c) Speedup and efficiency gain comparison.

These tightly coupled trade-offs call for a coordinated co-design of scoreboard capacity and GSAT granularity. We identify this balance point through DSE in Fig. 17 (a)(b).

Lightweight BS Scheduler. Built upon [15], PADE adopts a low-cost BS scheduler with a fundamentally different use of priority encoding (Fig. 12). Instead of instantiating multiple priority encoders in parallel as in [15], PADE exploits temporal reuse by time-multiplexing a single critical encoder across successive steps for index selection. Enabled by PADE's staggered QK-PU/V-PU pipeline, the added latency is effectively hidden, reducing priority-encoder overhead by 75%.

Specifically, the scheduler performs a runtime assessment of each Key bit-plane to determine whether zeros or ones dominate, and then forwards either the original or the bit-flipped column to a priority encoder. The priority encoder operates on the bit-plane in 5-bit windows, processing five consecutive bits per cycle. For example, at the first cycle, it receives $\{k_0,..,k_4\}$, followed by $\{k_1,..,k_5\}$ in the next, and so on. In each cycle, the encoder detects the position of the first bit "1" within each 5-bit window. If detected, the bit is masked, and the remaining bits are propagated to the next cycle. Otherwise, if no bit "1" is present, the encoder will pull down the signal $\mathbb{V}_x$ to disable the corresponding ANDer in the bit-wise PE lanes, as depicted in Fig. 11 (b).

E. Reuse-Aware Reorder Scheduling (RARS)

Due to attention sparsity, the remaining scores are distributed irregularly. For example, in Fig. 13 (a), the 0-th score row S_0 retains elements at positions 0-3, which are multiplied with the corresponding V_0-V_3 vectors (refer to Fig. 3).

However, a naive design leads to redundant V-vector memory accesses during the computation of $\mathbf{S} \times \mathbf{V}$. As shown in Fig. 13 (a), assuming each PE row in V-PU processes two V vectors per round, a naive left-to-right execution computes the dot products (S_0,V_0V_1), (S_1,V_2V_3), (S_2,V_4V_5), (S_3,V_2V_3) in round 0, followed by (S_0,V_2V_3), (S_1,V_4V_7), (S_2,V_6V_7), (S_3,V_4V_7) during round 1. Due to data reuse inefficiency, some shared V vectors must be reloaded, resulting in a total of 11 V-vector memory accesses.

To implement the ISTA strategy efficiently, we propose reuse-aware reorder scheduling (RARS) to reduce redundant memory access. As shown Fig. 13 (d), V_2 and V_3 are shared among three scores: S_0, S_1 and S_3, making them are prioritized for initial scheduling. Then, RARS selects V_5 and V_6, which are used exclusively by the remaining score S_2. As a result, V_2, V_3, V_5 and V_6 are grouped for execution in round 0. Such greedy search continues until all scores are allocated adequate Vs. As depicted in Fig. 13 (e), compared to the default left-to-right computation order, RARS reduces 30% memory access.

We design an efficient scheduler to implement RARS. As shown in Fig. 13 (c), condition statements and control logic are handled by an FSM controller. A single-port read-write ID buffer, indexed by score-derived bitmasks, stores the corresponding V-vector IDs. For example, V_5 and V_6, used exclusively by S_2, are stored in buffer-0010. Guided by RARS logic, the FSM retrieves the buffer entries and dispatches them to the issuing FIFO in an optimized execution order.

Authorized licensed use limited to: University of Electronic Science and Tech of China. Downloaded on October 31,2026 at 10:33:52 UTC from IEEE Xplore. Restrictions apply.

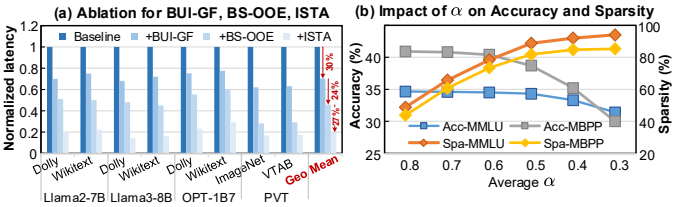


Fig. 16. (a) Latency reduction for BUI-GF, BS-OOE and ISTA. (b) Exploring the trade-off between accuracy and sparsity.

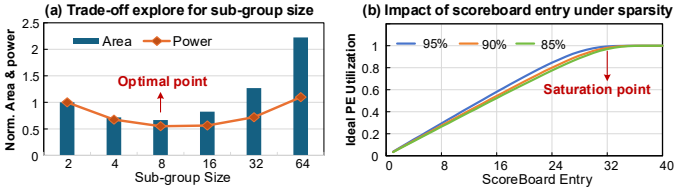


Fig. 17. Exploration of optimal (a) Sub-group size. (b) Scoreboard entry.

VI. Evaluation

A. Experimental Setup

Baseline comparisons: We compare PADE with five SOTA attention accelerators: Sanger [82], Spatten [119], Energon [153], DOTA [96], SOFA [122]. For fair comparison, all designs are normalized to a 28nm process and evaluated under identical conditions: PE arrays occupy the same area as PADE and work in 800 MHz, on-chip SRAM is set to 352kB, and peak HBM bandwidth is 256 GB/s, with 4 pj/bit [87].

Benchmarks: We evaluate PADE on several representative Transformer models across NLP and CV tasks. For NLP tasks, Llama2-7B [116], Llama3-8B [38], Qwen7B [6], Bloom1B7 [66] and OPT1B3 [150], for six tasks.These tasks include language modeling (Wikitext-2 (S=2k) [84], Wikilingua (S=2k) [29], Winogrande (S=0.25k) [101]), language understanding (MMLU, S=0.5k) [48], code generation MBPP (S=1k) [4], long context processing dolly (S=15k) [18]. For CV tasks, we choose ViT-L/16 (S=576) [28] and PVT (S=3k) [133] on ImageNet-1k [24] and JFT [112] classification.

Quantization Accuracy. All pre-trained models are sourced from Pytorch [91] and HuggingFace [137]. INT8 baselines derived via post-training quantization, where only the weights and activations (QKVs) are quantized to INT8, while non-linear operators (e.g., softmax) remain in FP16 precision. As shown in Table. II, the INT8 baseline incurs less than a 1% average accuracy drop from FP16, thus confirming its validity.

Hardware Evaluation: Table III lists the hardware configuration of PADE. We implement the RTL design for PADE and utilize Synopsys DC on TSMC 28nm CMOS technology to estimate the logic area and power. The power, area, and read/write bandwidth of on-chip SRAM buffers are estimated through CACTI [85]. Off-chip HBM modeling involves simulating access patterns and row activation under various data layouts (Fig. 22), capturing HBM's burst behavior. We derive memory latency from Ramulator [63], and estimate IO power following the methodology in [11], [2], [136]. We extract each stage's cycles by simulating the RTL with Verilator [108], and use a custom cycle-level simulator to evaluate performance.

GPU comparison We benchmark on an Nvidia H100 using SOTA TensorRT-LLM [86] with FlashAttention3 [102]. **To exclude the software overhead**, we measure execution time with *cudaEvent*, isolating GPU execution from CPU interference. The GPU is dedicated during testing, and large batch sizes are used to amortize data transfer costs. Non-computational phases are excluded using *nvprof*. Power is measured via *nvidia-smi*; dynamic power is computed as the difference between active and idle states. Each experiment is run 2k times, discarding the top and bottom 15% before averaging.

GPU test configurations: We test all different datasets with their allowed sequence lengths ranging from 0.25k to 15k. For example, MMLU (0.5k) and Dolly (15k). We measure the total inference latency, including the prefill and decoding. Specific prefill and decoding lengths are determined by the dataset itself. For batch size, we will select the configuration from [8, 128] that maximizes GPU computational efficiency based on the sequence length of each dataset.

B. Algorithm Performance

Algorithm settings: INT8 models serve as the accuracy baseline, and α (Eq.(4)) is adjusted in 0.1 increments to evaluate the accuracy and overhead for each benchmark. Two PADE configurations are evaluated: standard (0% loss), aggressive (1% loss).

Fig. 15 (a)(b) compares the accuracy of three representative software-only sparse attention methods, two predictor-free works (SpAtten [119], DTATrans [145]), and PADE. The *Sparsity Level* denotes the ratio between the computation cost of sparse execution (prediction + actual computation) and that of dense execution. StreamingLLM [141] adopts a static sparsity pattern—retaining only the initial and recent tokens, while MInference [57], and DoubleSparsity [144] rely on runtime sparsity prediction. Four key observations are made: **1)** StreamingLLM performs the worst, due to its lack of adaptivity in capturing input-dependent sparsity patterns. **2)** MInference improves accuracy by combining dynamic prediction with predefined sparsity patterns but remains limited by restricted pattern diversity. **3)** DoubleSparsity introduces a more flexible dynamic sparsity mechanism and integrates channel sparsity to reduce prediction overhead. However, as its prediction computation and memory access cannot be reused in subsequent steps, it suffers from inefficiency, yielding slightly lower accuracy than PADE at the same sparsity level. **4)** For SpAtten and DTATrans, comparable accuracy to PADE can be achieved only after fine-tuning. This is mainly because both methods eliminate the predictor by using the previous layer's attention distribution to guide pruning in the current layer, which introduces significant errors without fine-tuning. In contrast, PADE achieves the best performance across all settings, leveraging its fine-grained sparsity prediction and efficient reuse of both computation and memory access.

Fig. 15(c) further compares PADE (software–hardware co-optimization) with the software-only methods under the same 1% accuracy loss. PADE achieves an average 5.2× speedup and 10.4× improvement in energy efficiency. Moreover,

Authorized licensed use limited to: University of Electronic Science and Tech of China. Downloaded on October 31,2026 at 10:33:52 UTC from IEEE Xplore. Restrictions apply.

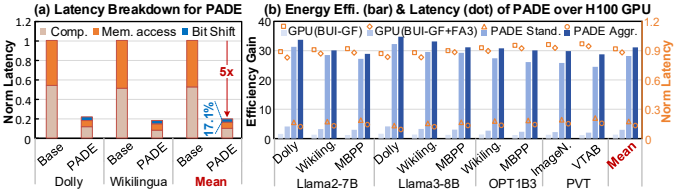


Fig. 18. (a) Breakdown for the overhead of bit shifting. (b) Latency and Efficiency gain of PADE over Nvidia H100 GPU.

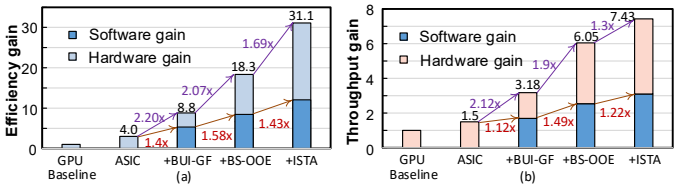


Fig. 19. Energy efficiency and throughput gain breakdown.

PADE's advantage becomes more pronounced as the sequence length increases, since longer sequences amplify the overhead of sparsity prediction. In such case, PADE's stage-fusion and reuse mechanisms effectively eliminate this prediction overhead, leading to superior scalability and efficiency.

Computation Reduction. Fig. 14 compares the computation reduction across accelerators with 0% accuracy loss. Spatten, without retraining, yields the lowest reduction and serves as the baseline. Energon enhances performance via progressive precision prediction, achieving a 32% reduction, and outperforming coarse-grained prediction accelerators like Sanger and DOTA. However, it lacks computation reuse. SOFA mitigates this by using log-domain differential leading-one computation, but still relies on an added predictor, limiting its reduction to 45%. In contrast, PADE eliminates the need for a predictor by leveraging fine-grained bit-level early termination and bit reuse, achieving a reduction of up to 71.6%.

Memory Access Reduction. As shown in Fig. 14, Sanger, which employs an extra 4-bit MSB predictor, serves as the baseline. DOTA adopts low-rank approximation but fails to mitigate prediction bitwidth overhead. Energon and Spatten* (with finetune) partially alleviate this via mixed precision, but still rely on an extra predictor, limiting their reduction to 21% and 42%, respectively. SOFA reduces memory overhead via cross-stage tiling yet remains constrained by the extra predictor. In contrast, PADE achieves an average memory reduction of 75.8% across both long- and short-sequence tasks, attributed to the BSF strategy. When compared to Spatten without fine-tune, PADE achieves 3.4× higher memory reduction.

C. Design of Architecture Parameters

The key architectural parameters in PADE are decided by detailed workload profiling and DSE exploration.

(1) Computational throughput: The INT-8 throughput ratio between QK-PU and V-PU is set to 8:1 (see Table III), derived from the typical QK-to-SV computation ratio observed in LLM workloads due to sparsity.

(2) Buffer Sizes: A 320 KB KV buffer and a 32 KB query buffer are allocated, sufficient for 12.8k tokens under typical sparsity ratio of 0.2 (64-d embedding) and 256 queries, respectively. These configurations balance on-chip storage efficiency and multi-phase processing for longer sequences.

(3) Optimal Sub-group Size. We conduct a DSE to determine the optimal PE group size (Recall §V-D). As shown in Fig. 17 (a), a sub-group size of 8 minimizes area and power overhead, which is therefore adopted in our PADE accelerator.

(4) Optimal Scoreboard Size. A larger scoreboard can cache more partial sums, which improves PE utilization but increases area overhead. Fig. 17 (b) shows that PE utilization saturates at around 32 entries. This indicates that, at this point, memory access and computation rates are balanced, preventing PE computation from being blocked. Therefore, PADE adopts a 32-entry scoreboard, as in Table III.

D. Architecture Evaluation

Ablation. We conduct an ablation study to evaluate the latency reduction of BUI-GF, BS-OOE, and ISTA against a baseline dense attention accelerator derived from PADE, but without sparse processing modules. As shown in Fig. 16 (a), BUI-GF reduces average latency by 30%, mainly by predictor-free token sparsity. Further, BS-OOE realizes 24% latency reduction via improved hardware utilization. Finally, ISTA achieves 27% decrease, via efficient tiling and data reuse.

Accuracy & Sparsity Trade-off. The BUI-GF may affect accuracy, as it introduces a parameter α to pruning KVs (§IV-A). Fig. 16 (b) shows the impact of α on accuracy and sparsity using LLaMA2-7B on MMLU (reasoning) and MBPP (generation). Overall, a smaller α results in more aggressive pruning, decreasing accuracy but increasing sparsity. There are some key observations: For generation tasks (MBPP), accuracy drops noticeably when $\alpha<0.6$. In contrast, for reasoning tasks (MMLU), the model is more tolerant to pruning, with accuracy degrading evidently only when $\alpha<0.5$. This may be because reasoning tasks rely on vital tokens for inference, resulting in higher token redundancy. On the other hand, the sparsity gains begin to diminish when $\alpha<0.5$, likely due to overly aggressive pruning, which harms crucial tokens and limits further sparsity. Therefore, to strike a balanced trade-off between accuracy and sparsity, we empirically set α within the range of 0.5–0.6.

Bit-serial overhead. Fig.18 (a) profiles latency overhead between the PADE architecture with value-level INT8 computation (baseline) and the bit-level PADE design. The 17% bit-shifting overhead is outweighed by a 5× latency reduction, validating the effectiveness of bit-level optimizations.

Comparison with GPU.Fig. 18 (b) compares the latency and energy efficiency of PADE with H100 GPU across various benchmarks. As shown, even with sparsity detection, BUI-GF enables only an average of 8% latency reduction and 1.3× efficiency gain on the GPU. Incorporating FlashAttention3 improves these figures to 14% latency reduction and 3.1× efficiency gain through memory-access reduction via tiling, but the improvement remains limited. This is because GPUs cannot leverage fine-grained bit-grained early termination or bit sparsity, nor can they efficiently execute bit-wise out-of-order computation. By contrast, PADE achieves an average 78% utilization thanks to dedicated hardware support, such as scoreboard PE, lightweight ANDer tree, and an efficient

Authorized licensed use limited to: University of Electronic Science and Tech of China. Downloaded on October 31,2026 at 10:33:52 UTC from IEEE Xplore. Restrictions apply.

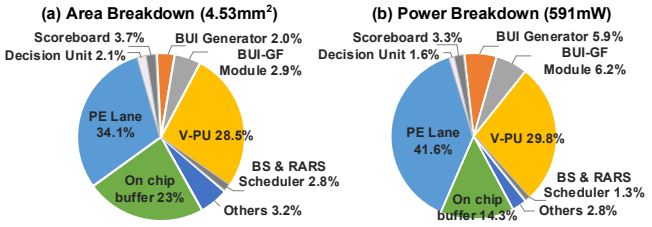


Fig. 20. Area/Power of PADE at TSMC 28nm, 800MHz.

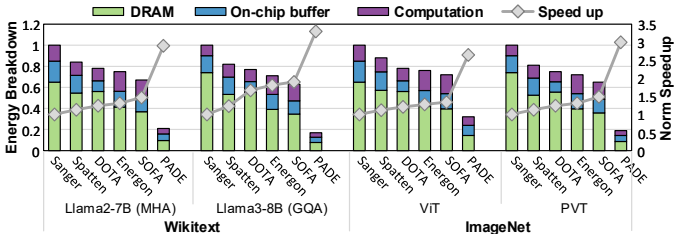


Fig. 21. Speedup and energy breakdown comparisons.

pipeline between QK-PU and V-PU. Overall, PADE standard and aggressive achieve an average 5.8×/7.4× latency reductions and 28.2×/31.1× energy efficiency gains, respectively.

Efficiency gain breakdown. Fig. 19 (a) gives the energy efficiency breakdown. With a dedicated ASIC and customized datapath, PADE achieves a 4.0× gain over the GPU baseline. While the token sparsity-leveraged BUI-GF theoretically reduces computation by 3.7×, practical efficiency improves by only 1.4×. This is due to redundantly loading repeated bit planes. After adding the dedicated scoreboard-based result reusable PE lane, the performance jumps by 2.2×. Similarly, directly applying the BS-OOE scheme and tiling-target ISTA scheme yields only 1.58× and 1.43× efficiency gain. This is due to mismatched computational granularity and the presence of unused V vectors, which ultimately lead to severe resource underutilization. By contrast, deploying tailored engines can further bring 2.07× and 1.69× efficiency gain effects.

Area and Power. Fig. 20 presents the area and power breakdown of PADE. Occupying 4.53 mm^2 and consuming 591 mW, PADE achieves a peak energy efficiency of 11.36 TOPS/W. The added BUI Generator and BUI-GF modules adaptively respond to attention distribution for token pruning, incurring only 4.9% area and 12.1% power overhead. Also, integrating the Scoreboard and Decision Unit into PE lanes enables stage fusion with just 5.8% area and 4.9% power cost. Despite the modest overhead, eliminating the sparsity predictor and reducing off-chip memory access yield notable speedup and efficiency gains. As depicted in Fig.19 (b), compared to baseline without sparse processing modules, software-hardware co-design BUI-GF, BS-OOE and ISTA bring 2.12×, 1.9× and 1.3× throughput gain, respectively. In summary, PADE represents a deliberate tradeoff, achieving substantial efficiency gains with minimal resource overhead.

Data Layout. In PADE, the data layout is carefully co-designed across the off-chip DRAM and on-chip SRAM to maximize memory bandwidth utilization. As depicted in Fig. 22 above, ***K is bank-interleaved along the bit dimension***, meaning that each DRAM bank stores a distinct bit-plane of the K tensor, enabling efficient bit-plane access when performing bit-level computations. In contrast, ***Q and V are bank-interleaved along the hidden (H) dimension*** such that 8-bit data is read continuously. When the data are fetched into the on-chip SRAM, the layout is reorganized to match the PE access pattern. In the Q,V SRAM, each row primarily stores different bits of the same element, whereas in the K SRAM, each row stores the same bit plane (e.g., MSB) from multiple elements across the hidden dimension.

E. Comparison to SOTA Accelerators

Fig. 21 compares the throughput and energy efficiency of PADE with five SOTA attention accelerators. Energy overhead is decomposed into computation, on-chip buffer, and off-chip memory. All existing designs rely on additional predictors to estimate attention sparsity. While these works partially reduce computational overhead, the inclusion of additional sparsity predictors limits their efficiency gains. Moreover, most designs fail to reduce off-chip memory access costs, leading to DRAM consistently accounting for over 65% of total energy. Two key observations are made: **1)** Llama2-7B vs. Llama3-8B: PADE achieves greater performance gains when GQA is adopted, as the scoreboard-based PE enhances key reuse across heads. **2)** ViT vs. PVT: PADE's acceleration advantage grows with longer sequences, as higher sparsity amplifies the predictor overhead in conventional designs, causing performance degradation. In contrast, PADE's predictor-free architecture avoids this overhead entirely. Overall, PADE achieves the highest performance across all workloads, achieving average speedups of 3×, 2.2×, 1.9× over Sanger, DOTA and SOFA, respectively, along with energy savings of 5.1×, 4.3×, 3.4×.

PE Utilization. Fig. 23 (a) presents a detailed breakdown of execution time across varying numbers of PE lanes to illustrate load balance effects. To better show PADE's advantages, we compare it against a SOTA bit accelerator, BitWave [106], which leverages bit-flipping to enhance bit-plane sparsity. Since each PE lane integrates multiple bit-serial multipliers, intra-PE stall arises when certain multipliers handle more effective bits. Inter-PE install, in contrast, results from variations in bit sparsity across different key vectors and waiting time for DRAM access. As the number of PE lanes increase,

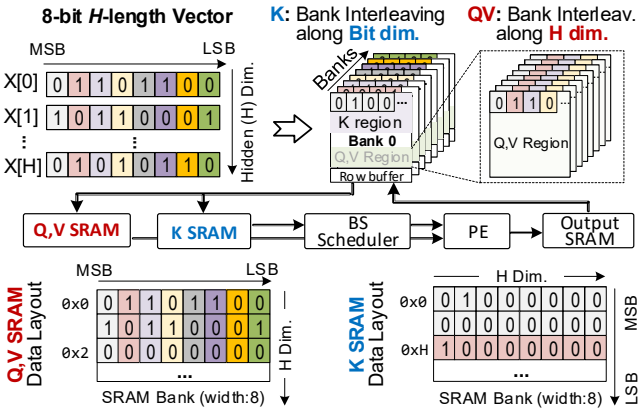


Fig. 22. DRAM, SRAM layouts and overall dataflow of PADE.

Authorized licensed use limited to: University of Electronic Science and Tech of China. Downloaded on October 31,2026 at 10:33:52 UTC from IEEE Xplore. Restrictions apply.

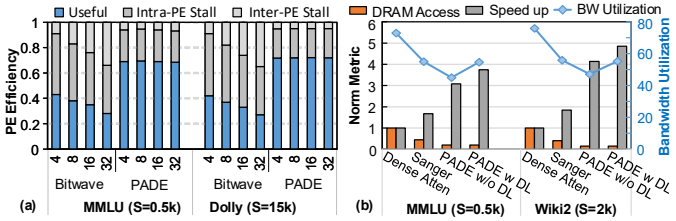


Fig. 23. (a) Breakdown of execution cycles versus the number of PE lanes. (b) Bandwidth utilization, DRAM access, and speedup comparisons.

BitWave suffers from greater intra- and inter-PE imbalance, as it fails to perform memory optimization and only exploits bit-0 sparsity, which exhibits large variability. In comparison, PADE adopts a more balanced bit sparsity distribution and effective memory-latency hiding, thereby achieving around 30% higher PE utilization.

DRAM Bandwidth (BW) Utilization. Fig. 23 (b) compares DRAM access, speedup, and bandwidth (BW) utilization of different attention accelerators on the MMLU and Wikitext2 workloads. In this context, *Dense Attention* refers to the method without sparse computation, while *Sanger* employs a coarse-grained 4-bit value for sparse prediction. *PADE w/o DL* represents PADE's fine-grained bit prediction without data layout optimization in DRAM and SRAM. *PADE w DL* refers to PADE with customized bit-oriented data layout, as shown in Fig. 22. Compared to the dense version, although PADE's bit-grained sparsity lowers DRAM bandwidth utilization by around 30%, memory access decreases over 6.7×, resulting in an average 3.4× speedup. After incorporating the bit-oriented data layout, the BW utilization improves to 58% due to higher row buffer hits, achieving a speedup of 4.3×.

F. Discussion for Deployment and Scalability

System Integration. PADE functions as a co-processor working collaboratively with the GPU. The GPU handles dense computations such as QKV projection and FFN, while PADE accelerates attention via sparsity. As depicted in Fig. 24 (a), both processors execute instructions issued by the host CPU and share the device memory, enabling direct data exchange without additional transfers. During the K generation, GPU executes an extra data conversion operation to store K in HBM with a bit-plane-first layout, as shown in Fig. 22.

Fig. 24 (b) compares the overall execution timeline for the PADE-equipped GPU (P-G) system with the original GPU-only system. As shown, the operations concerning two successive input sequences (I_0 and I_1) are interleaved on both

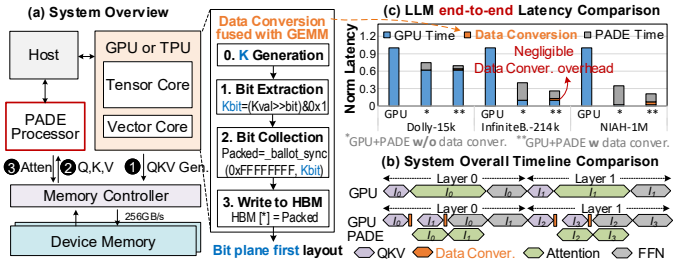


Fig. 24. Conceptual integration of PADE with a GPU-like NN accelerator.

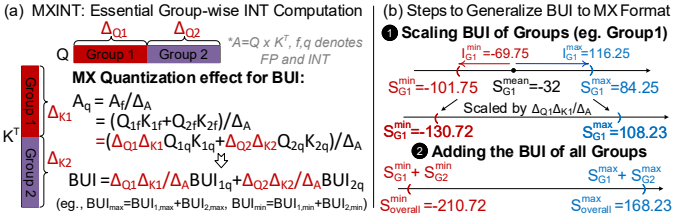


Fig. 25. Compatibility of the BUI-GF mechanism with the MX format.

GPU and PADE processors, greatly improving the system throughput. Fig. 24 (c) quantifies this speedup, showing that at 214k scenarios, the P-G system achieves a 2.1× speedup. However, without a customized data layout, redundant memory accesses limit performance gains. After incorporating the bit-oriented data layout, despite a latency increase of less than 2%, an additional 1.9× speedup is achieved.

Extension for MXINT. The micro-scale format performs fine-grained quantization along the channel dimension by grouping data into 32-element segments [98]. PADE ensures compatibility by applying group-wise scaling to the bit uncertain interval (BUI) (§IV-A). As shown in Fig. 25 (a), when processing 64-length Q and K vectors, the micro-format partitions them into two 32-element groups and quantizes each group using calibration-derived factors (Δ), resulting in group-wise BUI scaling. As depicted in Fig. 25 (b), PADE achieves compatibility via two steps. ❶ First, it performs bitwise serial multiplication within each 32-length group, with the BUI derived from the strategy in §IV-A. The resulting BUI is then expanded using the quantization factor, eg., $\Delta_{Q1}\Delta_{K1}/\Delta_A$. ❷ Finally, the max and min BUI values across all groups are aggregated to compute the overall BUI. This method can be extended to dot products of arbitrary length.

Extension for FP formats. Prior studies [105], [140], [152] have demonstrated that the K and V tensors are highly amenable to quantization because the softmax normalization in self-attention naturally suppresses quantization noise. This property enables safe quantization to INT8 or even INT4 with negligible accuracy degradation. Motivated by this, when queries operate in FP format, PADE converts the INT-FP computation into a bit-serial form through exponent alignment, following methodologies adopted in prior works [14], [54], [32]. The resulting bit-serial execution is fully compatible with PADE's existing processing mechanism.

Diverse Quantizations. To assess PADE's adaptability across quantization strategies and bit-widths, we extend both PADE and SOFA to QAT/PTQ INT8 and INT4 scenarios, as shown in Fig.26 (a). There are two key observations: (1) Compared to PTQ8, QAT8 leads to an 6% increase in energy consumption for SOFA. This is due to the more uniform data distribution under QAT, which reduces sparsity and renders SOFA's predictor largely ineffective. In contrast, PADE also shows a slight energy increase, but the overhead remains negligible, as it avoids the use of any explicit predictor. (2) When the bit-width is reduced to 4 bits, SOFA's energy efficiency gain drops significantly, as the predictor becomes the dominant source of power consumption under low-precision settings. In

Authorized licensed use limited to: University of Electronic Science and Tech of China. Downloaded on October 31,2026 at 10:33:52 UTC from IEEE Xplore. Restrictions apply.

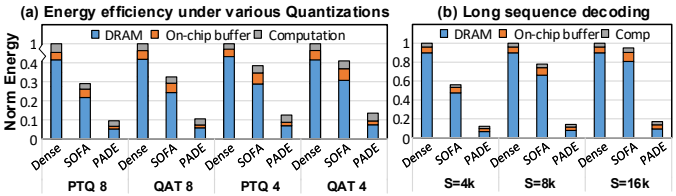


Fig. 26. Efficiency with (a) diverse quantizations, (b) long sequence decoding.

contrast, PADE's energy gain decreases by only 2%, thanks to its predictor-free design that avoids this overhead.

Ultra-long Sequence Decoding. In long-sequence decoding, memory access optimization is critical due to the lack of data reuse, making it a key indicator of a design's memory efficiency. As shown in Fig. 26 (b), in decoding, DRAM access accounts for over 85% of total power overhead across all designs, due to the autoregressive nature of the workload. Notably, compared to the dense version, SOFA's energy consumption increases significantly with sequence length—rising by nearly 40% from 4K to 16K tokens. This is because SOFA must load all key vectors corresponding to the predicted sequence at each decoding step, making the predictor's overhead grow rapidly. In contrast, PADE shows only a modest increase of about 5% over the same sequence range, thanks to its predictor-free architecture. **These results collectively demonstrate the advantages of PADE across diverse scenarios.**

G. Limitations and Future Direction

While PADE significantly advances sparse-attention acceleration by eliminating the sparsity predictor through unified bit-serial stage fusion, several limitations remain for future works: (1) As the sequence length of large language models (LLMs) continues to scale, the need for distributed attention becomes inevitable. Extending PADE to distributed scenarios, especially in emerging wafer-scale architectures [127], [129], [131], [46], [51], [114], remains an open problem. (2) Beyond single-bit granularity, exploring multi-bit stage fusion may provide a more favorable efficiency–accuracy trade-off, which represents a promising direction for future PADE enhancements.

VII. Related Works

Efficient Attention Accelerators. Numerous attention accelerators [124], [125], [126], [70], [42], [43], [96], [31], [145], [50], [78], [122], [134], [33], [147], [75], [82], [119], [153], [94], [104], [30], [8], [151], [117], [148], [104] have been proposed. Early works, such as A^3 [42] and ELSA [43] accelerate computation via approximation techniques. Recent efforts have shifted to jointly optimizing computation and memory. Energon [153] and SOFA [122] adopt fine-grained filtering and FlashAttention-optimized tiling techniques to alleviate memory overhead. However, they still rely on extra sparsity predictors, which become de facto latency and power bottlenecks after sparsification. PADE is the first to explicitly identify and address this issue. It removes the need for external predictors by fine-grained, bit-serial computation, while leveraging efficient bit-level early termination and reuse.

Neural network (NN) accelerator with sparsity. Numerous accelerators [130], [49], [132], [36], [3], [23], [93], [39], [33], [45], [69], [71], [74], [75], [79], [92], [99], [100], [118], [142], [37], [139], [138], [107] exploit sparsity to accelerate NN inference. General-purpose sparse tensor accelerators [111], [83], [65], [47], [61], [13] support operations on sparse fully connected layers, but most of them target pre-trained, statically sparse weights. By contrast, PADE targets attention dynamic sparsity, which requires on-the-fly prediction, making zero-based sparsity methods ineffective. While some recent works explore activation sparsity [55] or combine weight and activation sparsity [138], [52], [135], they still rely on zero-based sparsity, failing to address the argmax sparsity, which is the PADE targets.

Bit-serial computing accelerators. Prior works [128], [53], [44], [143], [58], [1], [22], [80], [103], [67], [21], [36], [71], [88], [146], [81], [60], [72] accelerates NNs by exploiting bit-level sparsity [1], [22], [81], [103], [143] or dynamically reducing bit-width [36], [71], [88], [146]. However, these techniques are difficult to apply directly to sparse attention, as they typically rely on offline weight preprocessing, while sparse attention demands real-time, token-wise prediction. In addition, the highly dynamic and fine-grained bit-level prediction leads to substantial utilization inefficiencies, further limiting their effectiveness on attention acceleration. In contrast, PADE adopts a lightweight runtime pruning strategy, BUI-GF, and maximizes hardware efficiency through BS-OOE, a memory-latency-hiding execution technique.

VIII. Conclusion

We propose PADE, a software-hardware co-design for dynamic sparse attention without relying on traditional sparsity predictors. Through bit-level early termination, dual-sided bit sparsity, dedicated bit-wise out-of-order execution, and an optimized tiling dataflow, PADE achieves up to 31.1× energy efficiency over H100 and 5.1× over SOTA accelerator Sanger.

Acknowledgments

We sincerely thank the reviewers for highlighting the importance of addressing memory issues at the system level. This work was supported in part by the National Science and Technology Major Project under Grant 2022ZD0115200; in part by the NSFC under Grant 62125403, Grant U24A20234, Grant 92464302 and Grant U24B20164; in part by the Beijing S&T Project Z251100008425010; in part by Shanghai Municipal Science and Technology Major Project; the Natural Science Foundation of Jiangsu Province Basic Research Program under Grant BK20243042; in part by the Beijing National Research Center for Information Science and Technology; in part by the Northern IC Technology Innovation Center (Beijing) Co., Ltd under Grant QYJS20232801B; and in part by the Beijing Advanced Innovation Center for Integrated Circuits.

References

[1] J. Albericio, A. Delmás, P. Judd, S. Sharify, G. O'Leary, R. Genov, and A. Moshovos, "Bit-pragmatic deep neural network computing," in *Proceedings of the 50th annual IEEE/ACM international symposium on microarchitecture*, 2017, pp. 382–394.

Authorized licensed use limited to: University of Electronic Science and Tech of China. Downloaded on October 31,2026 at 10:33:52 UTC from IEEE Xplore. Restrictions apply.

[2] R. Andri, L. Cavigelli, D. Rossi, and L. Benini, "YodaNN: An ultra-low power convolutional neural network accelerator based on binary weights," in *Proceedings of the IEEE Computer Society Annual Symposium on VLSI (ISVLSI)*, 2016, pp. 236–241.

[3] B. Asgari, R. Hadidi, T. Krishna, H. Kim, and S. Yalamanchili, "ALRESCHA: A lightweight reconfigurable sparse-computation accelerator," in *Proceedings of the IEEE International Symposium on High Performance Computer Architecture (HPCA)*, 2020, pp. 249–260.

[4] J. Austin, A. Odena, M. Nye, M. Bosma, H. Michalewski, D. Dohan, E. Jiang, C. Cai, M. Terry, Q. Le, and S. Charles, "Program synthesis with large language models," *arXiv preprint arXiv:2108.07732*, 2021.

[5] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan, R. Child, A. Ramesh, D. M. Ziegler, J. Wu, C. Winter, C. Hesse, M. Chen, E. Sigler, M. Litwin, S. Gray, B. Chess, J. Clark, C. Berner, S. McCandlish, A. Radford, I. Sutskever, and D. Amodei, "Language models are few-shot learners," *Advances in neural information processing systems*, vol. 33, pp. 1877–1901, 2020.

[6] J. Bai, S. Bai, Y. Chu, Z. Cui, K. Dang, X. Deng, Y. Fan, W. Ge, Y. Han, F. Huang, B. Hui, L. Ji, M. Li, J. Lin, R. Lin, D. Liu, G. Liu, C. Lu, K. Lu, J. Ma, R. Men, X. Ren, X. Ren, C. Tan, S. Tan, J. Tu, P. Wang, S. Wang, W. Wang, S. Wu, B. XU, J. Xu, A. Yang, H. Yang, J. Yang, S. Yang, Y. Yao, B. Yu, H. Yuan, Z. Yuan, J. Zhang, X. Zhang, Y. Zhang, Z. Zhang, C. Zhou, J. Zhou, X. Zhou, and T. Zhu, "Qwen technical report," *arXiv preprint arXiv:2309.16609*, 2023.

[7] Y. Bai, J. Chen, J. Chen, W. Chen, Z. Chen, C. Ding, L. Dong, Q. Dong, Y. Du, and K. Gao, "Seed-ASR: Understanding diverse speech and contexts with LLM-based speech recognition," *arXiv preprint arXiv:2407.04675*, 2024.

[8] Z. Bai, P. Dangi, H. Li, and T. Mitra, "SWAT: Scalable and efficient window attention-based Transformers acceleration on FPGAs," in *Proceedings of the 61st ACM/IEEE Design Automation Conference*, 2024, pp. 1–6.

[9] I. Beltagy, M. E. Peters, and A. Cohan, "Longformer: The long-document Transformer," *arXiv preprint arXiv:2004.05150*, 2020.

[10] R. P. Brent and P. Zimmermann, *Modern computer arithmetic*. Cambridge University Press, 2010, vol. 18.

[11] L. Cavigelli and L. Benini, "Origami: A 803-GOp/s/W convolutional network accelerator," *IEEE Transactions on Circuits and Systems for Video Technology*, vol. 27, no. 11, pp. 2461–2475, 2016.

[12] K. K. Chang, A. Kashyap, H. Hassan, S. Ghose, K. Hsieh, D. Lee, T. Li, G. Pekhimenko, S. Khan, and O. Mutlu, "Understanding latency variation in modern dram chips: Experimental characterization, analysis, and optimization," in *Proceedings of the 2016 ACM SIGMETRICS International Conference on Measurement and Modeling of Computer Science*, 2016, pp. 323–336.

[13] Y. Chen, G. Xiao, F. Wu, Z. Tang, and K. Li, "tpSpMV: A two-phase large-scale sparse matrix-vector multiplication kernel for manycore architectures," *Information Sciences*, vol. 523, pp. 279–295, 2020.

[14] Y. Chen, A. F. AbouElhamayed, X. Dai, Y. Wang, M. Andronic, G. A. Constantinides, and M. S. Abdelfattah, "Bitmod: Bit-serial mixture-of-datatype LLM acceleration," in *2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 2025, pp. 1082–1097.

[15] Y. Chen, J. Meng, J.-s. Seo, and M. S. Abdelfattah, "Bbs: Bi-directional bit-level sparsity for deep learning acceleration," in *2024 57th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2024, pp. 551–564.

[16] R. Child, S. Gray, A. Radford, and I. Sutskever, "Generating long sequences with sparse transformers," *arXiv preprint arXiv:1904.10509*, 2019.

[17] H. Cho, D. Kim, S.-E. Hwang, and J. Park, "Sparc: Token similarity-aware sparse attention transformer accelerator via row-wise clustering," in *Proceedings of the 61st ACM/IEEE Design Automation Conference*, 2024, pp. 1–6.

[18] M. Conover, M. Hayes, A. Mathur, J. Xie, J. Wan, S. Shah, A. Ghodsi, P. Wendell, M. Zaharia, and R. Xin, "Free dolly: Introducing the world's first truly open instruction-tuned LLM," *Company Blog of Databricks*, 2023.

[19] G. M. Correia, V. Niculae, and A. F. Martins, "Adaptively sparse Transformers," *arXiv preprint arXiv:1909.00015*, 2019.

[20] J. Dass, S. Wu, H. Shi, C. Li, Z. Ye, Z. Wang, and Y. Lin, "Vitality: Unifying low-rank and sparse approximation for vision transformer acceleration with a linear taylor attention," in *2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2023, pp. 415–428.

[21] A. Delmas, P. Judd, S. Sharify, and A. Moshovos, "Dynamic stripes: Exploiting the dynamic precision requirements of activation values in neural networks," *arXiv preprint arXiv:1706.00504*, 2017.

[22] A. Delmas Lascorz, P. Judd, D. M. Stuart, Z. Poulos, M. Mahmoud, S. Sharify, M. Nikolic, K. Siu, and A. Moshovos, "Bit-tactical: A software/hardware approach to exploiting value and bit sparsity in neural networks," in *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems*, 2019, pp. 749–763.

[23] C. Deng, Y. Sui, S. Liao, X. Qian, and B. Yuan, "GoSPA: An energy-efficient high-performance globally optimized sparse convolutional neural network accelerator," in *Proceedings of the ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*, 2021, pp. 1110–1123.

[24] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, "Imagenet: A large-scale hierarchical image database," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2009, pp. 248–255.

[25] M. Denton and H. Schmit, "Direct spatial implementation of sparse matrix multipliers for reservoir computing," in *2022 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2022, pp. 1–11.

[26] T. Dettmers, M. Lewis, Y. Belkada, and L. Zettlemoyer, "Gpt3. int8 (): 8-bit matrix multiplication for transformers at scale," *Advances in Neural Information Processing Systems*, vol. 35, pp. 30 318–30 332, 2022.

[27] X. Dong, J. Bao, D. Chen, W. Zhang, N. Yu, L. Yuan, D. Chen, and B. Guo, "Cswin transformer: A general vision transformer backbone with cross-shaped windows," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2022, pp. 12 124–12 134.

[28] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby, "An image is worth 16x16 words: Transformers for image recognition at scale," *arXiv preprint arXiv:2010.11929*, 2020.

[29] C. C. Faisal Ladhak, Esin Durmus and K. McKeown, "WikiLingua: A new benchmark dataset for multilingual abstractive summarization," in *Findings of EMNLP, 2020*, 2020.

[30] H. Fan, T. Chau, S. I. Venieris, R. Lee, A. Kouris, W. Luk, N. D. Lane, and M. S. Abdelfattah, "Adaptable butterfly accelerator for attention-based NNs via hardware and algorithm co-design," in *Proceedings of the 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2022, pp. 599–615.

[31] Z. Fan, Q. Zhang, P. Abillama, S. Shoouri, C. Lee, D. Blaauw, H.-S. Kim, and D. Sylvester, "Taskfusion: An efficient transfer learning architecture with dual delta sparsity for multi-task natural language processing," in *Proceedings of the 50th Annual International Symposium on Computer Architecture*, 2023, pp. 1–14.

[32] C. Fang, M. Shi, R. Geens, A. Symons, Z. Wang, and M. Verhelst, "Anda: Unlocking efficient LLM inference with a variable-length grouped activation data format," in *2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 2025, pp. 1467–1481.

[33] C. Fang, A. Zhou, and Z. Wang, "An algorithm-hardware co-optimized framework for accelerating N:M sparse Transformers," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 30, no. 11, pp. 1573–1586, 2022.

[34] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh, "GPTQ: Accurate post-training quantization for generative pre-trained transformers," *arXiv preprint arXiv:2210.17323*, 2022.

[35] K. A. A. Fuad and L. Chen, "A survey on sparsity exploration in Transformer-based accelerators," *Electronics*, vol. 12, no. 10, p. 2299, 2023.

[36] A. Gondimalla, N. Chesnut, M. Thottethodi, and T. Vijaykumar, "SparTen: A sparse tensor accelerator for convolutional neural networks," in *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture*, 2019, pp. 151–165.

[37] A. Gondimalla, M. Thottethodi, and T. N. Vijaykumar, "Eureka: Efficient tensor cores for one-sided unstructured sparsity in DNN

Authorized licensed use limited to: University of Electronic Science and Tech of China. Downloaded on October 31,2026 at 10:33:52 UTC from IEEE Xplore. Restrictions apply.

inference," in *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture*, 2023, p. 324–337.

[38] A. Grattafiori, A. Dubey, A. Jauhri, A. Pandey, A. Kadian, A. Al-Dahle, A. Letman, A. Mathur, A. Schelten, A. Vaughan, A. Yang, A. Fan, A. Goyal, A. Hartshorn, A. Yang, A. Mitra, A. Sravankumar, A. Korenev, A. Hinsvark, A. Rao, A. Zhang, A. Rodriguez, A. Gregerson, A. Spataru, B. Roziere, B. Biron, B. Tang, B. Chern, C. Caucheteux, C. Nayak, C. Bi, C. Marra, C. McConnell, C. Keller, C. Touret, C. Wu, C. Wong, and C. Canton Ferrer, "The llama 3 herd of models," *arXiv preprint arXiv:2407.21783*, 2024.

[39] S. Gudaparthi, S. Singh, S. Narayanan, R. Balasubramonian, and V. Sathe, "CANDLES: Channel-aware novel dataflow-microarchitecture co-design for low energy sparse neural network acceleration," in *Proceedings of the IEEE International Symposium on high-performance computer architecture (HPCA)*, 2022, pp. 876–891.

[40] C. Guo, C. Wei, J. Tang, B. Duan, S. Han, H. Li, and Y. Chen, "Transitive array: An efficient GEMM accelerator with result reuse," in *Proc. IEEE Annu. Int. Symp. Comput. Archit. (ISCA)*, 2025, pp. 990–1004.

[41] R. Guo, Z. Yue, X. Si, T. Hu, H. Li, L. Tang, Y. Wang, L. Liu, M.-F. Chang, Q. Li, S. Wei, and S. Yin, "15.4 A 5.99-to-691.1 TOPS/W tensor-train in-memory-computing processor using bit-level-sparsity-based optimization and variable-precision quantization," in *2021 IEEE International Solid-State Circuits Conference (ISSCC)*, vol. 64. IEEE, 2021, pp. 242–244.

[42] T. J. Ham, S. J. Jung, S. Kim, Y. H. Oh, Y. Park, Y. Song, J.-H. Park, S. Lee, K. Park, J. W. Lee, and D.-K. Jeong, "A^3: Accelerating attention mechanisms in neural networks with approximation," in *Proceedings of the IEEE International Symposium on High Performance Computer Architecture (HPCA)*, 2020, pp. 328–341.

[43] T. J. Ham, Y. Lee, S. H. Seo, S. Kim, H. Choi, S. J. Jung, and J. W. Lee, "ELSA: Hardware-software co-design for efficient, lightweight self-attention mechanism in neural networks," in *Prceedings of the 48th ACM/IEEE Annual International Symposium on Computer Architecture (ISCA)*, 2021, pp. 692–705.

[44] M. Han, L. Wang, L. Xiao, H. Zhang, T. Cai, J. Xu, Y. Wu, C. Zhang, and X. Xu, "BitNN: A bit-serial accelerator for k-nearest neighbor search in point clouds," in *Proceddings of the ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*, 2024, pp. 1278–1292.

[45] E. Hanson, S. Li, H. Li, and Y. Chen, "Cascading structured pruning: Enabling high data reuse for sparse DNN accelerators," in *Proceedings of the 49th Annual International Symposium on Computer Architecture*, 2022, pp. 522–535.

[46] C. He, Y. Huang, P. Mu, Z. Miao, J. Xue, L. Ma, F. Yang, and L. Mai, "WaferLLM: Large language model inference at wafer scale," *arXiv preprint arXiv:2502.04563*, 2025.

[47] K. Hegde, H. Asghari-Moghaddam, M. Pellauer, N. Crago, A. Jaleel, E. Solomonik, J. Emer, and C. W. Fletcher, "ExTensor: An accelerator for sparse tensor algebra," in *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture*, 2019, pp. 319–333.

[48] D. Hendrycks, C. Burns, S. Basart, A. Zou, M. Mazeika, D. Song, and J. Steinhardt, "Measuring massive multitask language understanding," *arXiv preprint arXiv:2009.03300*, 2020.

[49] R. Hojabr, A. Sedaghati, A. Sharifian, A. Khonsari, and A. Shriraman, "SPAGHETTI: Streaming accelerators for highly sparse GEMM on FPGAs," in *Proceedings of the IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, 2021, pp. 84–96.

[50] S. Hong, S. Moon, J. Kim, S. Lee, M. Kim, D. Lee, and J.-Y. Kim, "DFX: A low-latency multi-FPGA appliance for accelerating Transformer-based text generation," in *Proceedings of the 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2022, pp. 616–630.

[51] Y. Hu, X. Lin, H. Wang, Z. He, X. Yu, J. Zhang, Q. Yang, Z. Xu, S. Guan, J. Fang, H. Shang, X. Tang, X. Dai, S. Wei, and S. Yin, "Wafer-scale computing: advancements, challenges, and future perspectives [Feature]," *IEEE Circuits and Systems Magazine*, vol. 24, no. 1, pp. 52–81, 2024.

[52] G. Huang, Z. Wang, P.-A. Tsai, C. Zhang, Y. Ding, and Y. Xie, "RM-STC: Row-merge dataflow inspired GPU sparse tensor core for energy-efficient sparse acceleration," in *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture*, 2023, pp. 338–352.

[53] D. Im, G. Park, Z. Li, J. Ryu, and H.-J. Yoo, "Sibia: Signed bit-slice architecture for dense DNN acceleration with slice-level sparsity exploitation," in *2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2023, pp. 69–80.

[54] J. Jang, Y. Kim, J. Lee, and J.-J. Kim, "FIGNA: Integer unit-based accelerator design for FP-INT GEMM preserving numerical accuracy," in *2024 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2024, pp. 760–773.

[55] J.-W. Jang, S. Lee, D. Kim, H. Park, A. S. Ardestani, Y. Choi, C. Kim, Y. Kim, H. Yu, H. Abdel-Aziz, J.-S. Park, H. Lee, D. Lee, M. W. Kim, H. Jung, H. Nam, D. Lim, S. Lee, J.-H. Song, S. Kwon, J. Hassoun, S. Lim, and C. Choi, "Sparsity-aware and re-configurable NPU architecture for Samsung flagship mobile SoC," in *Proceedings of the 48th Annual ACM/IEEE International Symposium on Computer Architecture (ISCA)*, 2021, pp. 15–28.

[56] JEDEC, *JESD235D: High Bandwidth Memory DRAM (HBM1, HBM2)*, JEDEC Standard JESD235D, 2021.

[57] H. Jiang, Y. Li, C. Zhang, Q. Wu, X. Luo, S. Ahn, Z. Han, A. H. Abdi, D. Li, C.-Y. Lin, Y. Yang, and Q. Lili, "MInference 1.0: Accelerating prefilling for long-context LLMs via dynamic sparse attention," *arXiv preprint arXiv:2407.02490*, 2024.

[58] P. Judd, J. Albericio, T. Hetherington, T. M. Aamodt, and A. Moshovos, "Stripes: Bit-serial deep neural network computing," in *Proceedings of the 49th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2016, pp. 1–12.

[59] C. Kachris, "A survey on hardware accelerators for large language models," *Applied Sciences*, vol. 15, no. 2, p. 586, 2025.

[60] D. Kam, M. Yun, S. Yoo, S. Hong, Z. Zhang, and Y. Lee, "Panacea: Novel DNN accelerator using accuracy-preserving asymmetric quantization and energy-saving bit-slice sparsity," in *Proc. IEEE Int. Symp. High Perform. Comput. Archit. (HPCA)*, 2025, pp. 701–715.

[61] K. Kanellopoulos, N. Vijaykumar, C. Giannoula, R. Azizi, S. Koppula, N. M. Ghiasi, T. Shahroodi, J. G. Luna, and O. Mutlu, "SMASH: Co-designing software compression and hardware-accelerated indexing for efficient sparse matrix operations," in *Proceedings of the 52nd annual IEEE/ACM international symposium on microarchitecture*, 2019, pp. 600–614.

[62] A. Katharopoulos, A. Vyas, N. Pappas, and F. Fleuret, "Transformers are RNNS: Fast autoregressive Transformers with linear attention," in *International conference on machine learning*. PMLR, 2020, pp. 5156–5165.

[63] Y. Kim, W. Yang, and O. Mutlu, "Ramulator: A fast and extensible DRAM simulator," *IEEE Computer architecture letters*, vol. 15, no. 1, pp. 45–49, 2015.

[64] N. Kitaev, Ł. Kaiser, and A. Levskaya, "Reformer: The efficient Transformer," *arXiv preprint arXiv:2001.04451*, 2020.

[65] Y. Kwon, Y. Lee, and M. Rhu, "TensorDIMM: A practical near-memory processing architecture for embeddings and tensor operations in deep learning," in *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture*, 2019, pp. 740–753.

[66] T. Le Scao, A. Fan, C. Akiki, E. Pavlick, S. Ilić, D. Hesslow, R. Castagné, A. S. Luccioni, F. Yvon, d. M. Gallé, Matthias Albert Villanova, , O. Ruwase, R. Bawden, P. O. Suarez, V. Sanh, H. Laurencon, Y. Jernite, J. Launay, M. Mitchell, and C. Raffel, "Bloom: A 176B-parameter open-access multilingual language model," *arXiv preprint arXiv:2211.05100*, 2022.

[67] J. Lee, C. Kim, S. Kang, D. Shin, S. Kim, and H.-J. Yoo, "UNPU: A 50.6 TOPS/W unified deep neural network accelerator with 1b-to-16b fully-variable weight bit-precision," in *Proceedings of IEEE International Solid-State Circuits Conference-(ISSCC)*, 2018, pp. 218–220.

[68] S. Lee, B. Kim, J. Park, and D. Jeon, "Clat: A clustering-based attention transformer accelerator for low-latency text generation in llms," *IEEE Transactions on Circuits and Systems I: Regular Papers*, 2025.

[69] J. S. Lew, Y. Liu, W. Gong, N. Goli, R. D. Evans, and T. M. Aamodt, "Anticipating and eliminating redundant computations in accelerated sparse training," in *Proceedings of the 49th Annual International Symposium on Computer Architecture*, 2022, pp. 536–551.

[70] B. Li, S. Pandey, H. Fang, Y. Lyv, J. Li, J. Chen, M. Xie, L. Wan, H. Liu, and C. Ding, "FTRANS: Energy-efficient acceleration of Transformers using FPGA," in *Proceedings of the ACM/IEEE International Symposium on Low Power Electronics and Design*, 2020, pp. 175–180.

[71] G. Li, W. Xu, Z. Song, N. Jing, J. Cheng, and X. Liang, "Ristretto: An atomized processing architecture for sparsity-condensed stream flow in

Authorized licensed use limited to: University of Electronic Science and Tech of China. Downloaded on October 31,2026 at 10:33:52 UTC from IEEE Xplore. Restrictions apply.

CNN," in *Proceedings of the 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2022, pp. 1434–1450.
[72] G. Li, S. Ye, C. Chen, Y. Wang, F. Yang, T. Cao, C. Liu, M. M. Sabry, and M. Yang, "Lut-dla: Lookup table as efficient extreme low-bit deep learning accelerator," *arXiv preprint arXiv:2501.10658*, 2025.
[73] J. Li, J. Xu, S. Huang, Y. Chen, W. Li, J. Liu, Y. Lian, J. Pan, L. Ding, and H. Zhou, "Large language model inference acceleration: A comprehensive hardware perspective," *arXiv preprint arXiv:2410.04466*, 2024.
[74] S. Li, E. Hanson, X. Qian, H. H. Li, and Y. Chen, "ESCALATE: Boosting the efficiency of sparse CNN accelerator with kernel decomposition," in *Proceedings of the 54th Annual IEEE/ACM International Symposium on Microarchitecture*, 2021, pp. 992–1004.
[75] Z. Li, S. Ghodrati, A. Yazdanbakhsh, H. Esmaeilzadeh, and M. Kang, "Accelerating attention through gradient-based learned runtime pruning," in *Proceedings of the 49th Annual International Symposium on Computer Architecture*, 2022, pp. 902–915.
[76] H. Lin, A. Zala, J. Cho, and M. Bansal, "Videodirectorgpt: Consistent multi-scene video generation via LLM-guided planning," *arXiv preprint arXiv:2309.15091*, 2023.
[77] L. Liu, Z. Qu, Z. Chen, F. Tu, Y. Ding, and Y. Xie, "Dynamic sparse attention for scalable transformer acceleration," *IEEE Transactions on Computers*, vol. 71, no. 12, pp. 3165–3178, 2022.
[78] S. Liu, P. C. Kuve, and A. Karanth, "Hsconn: Hardware-software co-optimization of self-attention neural networks for large language models," in *Proceedings of the Great Lakes Symposium on VLSI 2024*, 2024, pp. 736–741.
[79] Z.-G. Liu, P. N. Whatmough, Y. Zhu, and M. Mattina, "S2TA: Exploiting structured sparsity for energy-efficient mobile CNN acceleration," in *Proceedings of the IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, 2022, pp. 573–586.
[80] Y.-C. Lo and R.-S. Liu, "Bit-serial cache: Exploiting input bit vector repetition to accelerate bit-serial inference," in *Proceedings of the 60th ACM/IEEE Design Automation Conference (DAC)*, 2023, pp. 1–6.
[81] H. Lu, L. Chang, C. Li, Z. Zhu, S. Lu, Y. Liu, and M. Zhang, "Distilling bit-level sparsity parallelism for general purpose deep learning acceleration," in *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture*, 2021, pp. 963–976.
[82] L. Lu, Y. Jin, H. Bi, Z. Luo, P. Li, T. Wang, and Y. Liang, "Sanger: A co-design framework for enabling sparse attention using reconfigurable architecture," in *Proceedings of the 54th Annual IEEE/ACM International Symposium on Microarchitecture*, 2021, pp. 977–991.
[83] M. Mahmoud, I. Edo, A. H. Zadeh, O. M. Awad, G. Pekhimenko, J. Albericio, and A. Moshovos, "TensorDash: Exploiting sparsity to accelerate deep neural network training," in *Proceedings of the 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2020, pp. 781–795.
[84] S. Merity, C. Xiong, J. Bradbury, and R. Socher, "Pointer sentinel mixture models," in *Proceedings of the International Conference on Learning Representations*, 2016.
[85] N. Muralimanohar, R. Balasubramonian, and N. P. Jouppi, "CACTI 6.0: A tool to model large caches," *HP laboratories*, vol. 27, p. 28, 2009.
[86] Nvidia, "TensorRT-LLM," 2023, https://github.com/NVIDIA/TensorRT-LLM?tab=readme-ov-file.
[87] M. O'Connor, N. Chatterjee, D. Lee, J. Wilson, A. Agrawal, S. W. Keckler, and W. J. Dally, "Fine-grained DRAM: Energy-efficient DRAM for extreme bandwidth systems," in *Proceedings of the 50th Annual IEEE/ACM International Symposium on Microarchitecture*, 2017, pp. 41–54.
[88] A. Parashar, M. Rhu, A. Mukkara, A. Puglielli, R. Venkatesan, B. Khailany, J. Emer, S. W. Keckler, and W. J. Dally, "SCNN: An accelerator for compressed-sparse convolutional neural networks," *ACM SIGARCH computer architecture news*, vol. 45, no. 2, pp. 27–40, 2017.
[89] J. Park, M. Kang, Y. Han, Y.-G. Kim, J. Shin, and L.-S. Kim, "Token-picker: Accelerating attention in text generation with minimized memory transfer via probability estimation," in *Proceedings of the 61st ACM/IEEE Design Automation Conference*, 2024, pp. 1–6.
[90] N. Parmar, A. Vaswani, J. Uszkoreit, L. Kaiser, N. Shazeer, A. Ku, and D. Tran, "Image transformer," in *International conference on machine learning*. PMLR, 2018, pp. 4055–4064.
[91] A. Paszke, S. Gross, S. Chintala, G. Chanan, E. Yang, Z. DeVito, Z. Lin, A. Desmaison, L. Antiga, and A. Lerer, "Automatic differentiation in PyTorch," 2017.
[92] J. Pavon, I. V. Valdivieso, A. Barredo, J. Marimon, M. Moreto, F. Moll, O. Unsal, M. Valero, and A. Cristal, "VIA: A smart scratchpad for vector units with application to sparse matrix computations," in *Proceedings of the IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, 2021, pp. 921–934.
[93] E. Qin, A. Samajdar, H. Kwon, V. Nadella, S. Srinivasan, D. Das, B. Kaul, and T. Krishna, "Sigma: A sparse and irregular GEMM accelerator with flexible interconnects for DNN training," in *Proceedings of the IEEE International Symposium on High Performance Computer Architecture (HPCA)*, 2020, pp. 58–70.
[94] Y. Qin, Y. Wang, D. Deng, Z. Zhao, X. Yang, L. Liu, S. Wei, Y. Hu, and S. Yin, "FACT: FFN-attention co-optimized Transformer architecture with eager correlation prediction," in *Proceedings of the 50th Annual International Symposium on Computer Architecture*, 2023, pp. 1–14.
[95] J. Qiu, H. Ma, O. Levy, S. W.-t. Yih, S. Wang, and J. Tang, "Blockwise self-attention for long document understanding," *arXiv preprint arXiv:1911.02972*, 2019.
[96] Z. Qu, L. Liu, F. Tu, Z. Chen, Y. Ding, and Y. Xie, "DOTA: Detect and omit weak attentions for scalable Transformer acceleration," in *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, 2022, pp. 14–26.
[97] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, "Language models are unsupervised multitask learners," *OpenAI blog*, vol. 1, no. 8, p. 9, 2019.
[98] B. D. Rouhani, R. Zhao, A. More, M. Hall, A. Khodamoradi, S. Deng, D. Choudhary, M. Cornea, E. Dellinger, K. Denolf, S. Dusan, V. Elango, M. Golub, A. Heinecke, P. James-Roxby, D. Jani, G. Kolhe, M. Langhammer, A. Li, L. Melnick, M. Mesmakhosroshahi, A. Rodriguez, M. Schulte, R. Shafipour, L. Shao, M. Siu, P. Dubey, P. Micikevicius, M. Naumov, C. Verrilli, R. Wittig, D. Burger, and E. Chung, "Microscaling data formats for deep learning," *arXiv preprint arXiv:2310.10537*, 2023.
[99] A. Rucker, M. Vilim, T. Zhao, Y. Zhang, R. Prabhakar, and K. Olukotun, "Capstan: A vector RDA for sparsity," in *Proceedings of the 54th Annual IEEE/ACM International Symposium on Microarchitecture*, 2021, pp. 1022–1035.
[100] F. Sadi, J. Sweeney, T. M. Low, J. C. Hoe, L. Pileggi, and F. Franchetti, "Efficient SPMV operation for large and highly sparse matrices using scalable multi-way merge parallelization," in *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture*, 2019, pp. 347–358.
[101] K. Sakaguchi, R. L. Bras, C. Bhagavatula, and Y. Choi, "Winogrande: An adversarial Winograd schema challenge at scale," *Communications of the ACM*, vol. 64, no. 9, pp. 99–106, 2021.
[102] J. Shah, G. Bikshandi, Y. Zhang, V. Thakkar, P. Ramani, and T. Dao, "Flashattention-3: Fast and accurate attention with asynchrony and low-precision," *Advances in Neural Information Processing Systems*, vol. 37, pp. 68 658–68 685, 2024.
[103] S. Sharify, A. D. Lascorz, M. Mahmoud, M. Nikolic, K. Siu, D. M. Stuart, Z. Poulos, and A. Moshovos, "Laconic deep learning inference acceleration," in *Proceedings of the 46th International Symposium on Computer Architecture*, 2019, pp. 304–317.
[104] G. Shen, J. Zhao, Q. Chen, J. Leng, C. Li, and M. Guo, "SALO: An efficient spatial accelerator enabling hybrid sparse attention mechanisms for long sequences," in *Proceedings of the 59th ACM/IEEE Design Automation Conference*, 2022, pp. 571–576.
[105] Y. Sheng, L. Zheng, B. Yuan, Z. Li, M. Ryabinin, B. Chen, P. Liang, C. Ré, I. Stoica, and C. Zhang, "Flexgen: High-throughput generative inference of large language models with a single gpu," in *International Conference on Machine Learning*. PMLR, 2023, pp. 31 094–31 116.
[106] M. Shi, V. Jain, A. Joseph, M. Meijer, and M. Verhelst, "Bitwave: Exploiting column-based bit-level sparsity for deep learning acceleration," in *Proceedings of IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, 2024, pp. 732–746.
[107] J. H. Shin, A. Shafiee, A. Pedram, H. Abdel-Aziz, L. Li, and J. Hassoun, "Griffin: Rethinking sparse optimization for deep learning architectures," in *Proceedings of the IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, 2022, pp. 861–875.
[108] W. Snyder, "Verilator and SystemPerl," in *North American SystemC Users' Group, Design Automation Conference*, 2004.
[109] M. Song, K. Zhong, J. Zhang, Y. Hu, D. Liu, W. Zhang, J. Wang, and T. Li, "In-situ AI: Towards autonomous and incremental deep learning for IoT systems," in *2018 IEEE International Symposium on*

Authorized licensed use limited to: University of Electronic Science and Tech of China. Downloaded on October 31,2026 at 10:33:52 UTC from IEEE Xplore. Restrictions apply.

High Performance Computer Architecture (HPCA). IEEE, 2018, pp. 92–103.

[110] Z. Song, C. Qi, Y. Yao, P. Zhou, Y. Zi, N. Wang, and X. Liang, "Tsacc: An efficient t empo-s patial similarity aware acc elerator for attention acceleration," in *Proceedings of the 61st ACM/IEEE Design Automation Conference*, 2024, pp. 1–6.

[111] N. Srivastava, H. Jin, J. Liu, D. Albonesi, and Z. Zhang, "MatRaptor: A sparse-sparse matrix multiplication accelerator based on row-wise product," in *Proceedings of the 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2020, pp. 766–780.

[112] C. Sun, A. Shrivastava, S. Singh, and A. Gupta, "Revisiting unreasonable effectiveness of data in deep learning era," in *Proceedings of the IEEE international conference on computer vision*, 2017, pp. 843–852.

[113] W. Sun, Z. Zou, D. Liu, W. Sun, S. Chen, and Y. Kang, "Bit-balance: Model-hardware codesign for accelerating nns by exploiting bit-level sparsity," *IEEE Transactions on Computers*, vol. 73, no. 1, pp. 152–163, 2023.

[114] X. Tang, J. Hou, D. Jiang, T. Wei, J. Liu, J. Deng, H. Wang, Q. Yang, H. Shang, C. Li, Y. Hu, and S. Yin, "MoEntwine: Unleashing the potential of wafer-scale chips for large-scale expert parallel inference," *arXiv preprint arXiv:2510.25258*, 2025.

[115] Y. Tian, Y. Wang, B. Chen, and S. S. Du, "Scan and snap: Understanding training dynamics and token composition in 1-layer transformer," *Advances in neural information processing systems*, vol. 36, pp. 71 911–71 947, 2023.

[116] H. Touvron, L. Martin, K. Stone, P. Albert, A. Almahairi, Y. Babaei, N. Bashlykov, S. Batra, P. Bhargava, S. Bhosale, B. Dan, B. Lukas, C. F. Cristian, C. Moya, C. Guillem, E. David, F. Jude, F. Jeremy, F. Wenyin, F. Brian, G. Cynthia, G. Vedanuj, G. Naman, H. Anthony, H. Saghar, H. Rui, I. Hakan, and K. Marcin, "Llama 2: Open foundation and fine-tuned chat models," *arXiv preprint arXiv:2307.09288*, 2023.

[117] S. Tuli and N. K. Jha, "Acceltran: A sparsity-aware accelerator for dynamic inference with transformers," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 42, no. 11, pp. 4038–4051, 2023.

[118] S. Walia, B. V. Tej, A. Kabra, J. Devnath, and J. Mekie, "Fast and low-power quantized fixed posit high-accuracy DNN implementation," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 30, no. 1, pp. 108–111, 2021.

[119] H. Wang, Z. Zhang, and S. Han, "SpAtten: Efficient sparse attention architecture with cascade token and head pruning," in *Proceedings of the IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, 2021, pp. 97–110.

[120] H. Wang, H. Xu, Y. Wang, and Y. Han, "CTA: Hardware-software co-design for compressed token attention mechanism," in *2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2023, pp. 429–441.

[121] H. Wang, B. Cheng, X. Tan, X. You, and C. Zhang, "An efficient approximate expectation propagation detector with block-diagonal Neumann-series," *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 70, no. 3, pp. 1403–1416, 2023.

[122] H. Wang, J. Fang, X. Tang, Z. Yue, J. Li, Y. Qin, S. Guan, Q. Yang, Y. Wang, C. Li, Y. Hu, and S. Yin, "SOFA: A compute-memory optimized sparsity accelerator via cross-stage coordinated tiling," in *2024 57th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2024, pp. 1247–1263.

[123] H. Wang, Y. Ji, Y. Shen, W. Song, M. Li, X. You, and C. Zhang, "An efficient detector for massive MIMO based on improved matrix partition," *IEEE Transactions on Signal Processing*, vol. 69, pp. 2971–2986, 2021.

[124] H. Wang, H. Wang, S. Wei, Y. Hu, and S. Yin, "BitStopper: An efficient Transformer attention accelerator via stage-fusion and early termination," *arXiv preprint arXiv:2512.06457*, 2025.

[125] H. Wang, H. Wang, S. Wei, Y. Hu, and S. Yin, "LAPA: Log-domain prediction-driven dynamic sparsity accelerator for Transformer model," *arXiv preprint arXiv:2512.07855*, 2025.

[126] H. Wang, H. Wang, Z. Yue, J. Liu, T. Wei, S. W. Y. Hu, and S. Yin, "BETA: A bit-Grained Transformer attention accelerator With efficient early termination," *IEEE Transactions on Circuits and Systems II: Express Briefs*, 2025.

[127] H. Wang, Z. Wang, H. Wang, J. Hou, T. Wei, C. Li, Y. Hu, and S. Yin, "WATOS: Efficient LLM training strategies and architecture co-exploration for wafer-scale chip," in *Proceedings of the IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, 2026, accepted.

[128] H. Wang, Z. Wang, Z. Yue, Y. Long, T. Wei, J. Yang, Y. Wang, C. Li, S. Wei, Y. Hu, and S. Yin, "MCBP: A memory-compute efficient LLM inference accelerator leveraging bit-slice-enabled sparsity and repetitiveness," in *Proceedings of the 58th IEEE/ACM International Symposium on Microarchitecture*, 2025, pp. 1592–1608.

[129] H. Wang, T. Wei, Z. Wang, D. Jiang, Q. Yang, J. Liu, J. Hou, C. Li, J. Deng, Y. Hu, and S. Yin, "TEMP: A memory efficient physical-aware tensor partition-mapping framework on wafer-scale chips," in *Proceedings of the IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, 2026, accepted.

[130] H. Wang, W. Xu, Z. Zhang, X. You, and C. Zhang, "An efficient stochastic convolution architecture based on fast FIR algorithm," *IEEE Transactions on Circuits and Systems II: Express Briefs*, vol. 69, no. 3, pp. 984–988, 2021.

[131] H. Wang, Q. Yang, T. Wei, X. Yu, C. Li, J. Fang, G. Lu, X. Dai, L. Liu, S. Jiang, Y. Hu, and S. Yin, "TMAC: Training-targeted mapping and architecture co-exploration for wafer-scale chips," *Integrated Circuits and Systems*, 2024.

[132] H. Wang, Z. Zhang, X. You, and C. Zhang, "Low-complexity Winograd convolution architecture based on stochastic computing," in *2018 IEEE 23rd International Conference on Digital Signal Processing (DSP)*. IEEE, 2018, pp. 1–5.

[133] W. Wang, E. Xie, X. Li, D.-P. Fan, K. Song, D. Liang, T. Lu, P. Luo, and L. Shao, "Pyramid vision Transformer: A versatile backbone for dense prediction without convolutions," in *Proceedings of the IEEE/CVF international conference on computer vision*, 2021, pp. 568–578.

[134] Y. Wang, Y. Qin, D. Deng, J. Wei, Y. Zhou, Y. Fan, T. Chen, H. Sun, L. Liu, S. Wei, and S. Yin, "An energy-efficient Transformer processor exploiting dynamic weak relevances in global attention," *IEEE Journal of Solid-State Circuits*, vol. 58, no. 1, pp. 227–242, 2022.

[135] Y. Wang, C. Zhang, Z. Xie, C. Guo, Y. Liu, and J. Leng, "Dual-side sparse tensor core," in *Proceedings of the 48th Annual ACM/IEEE International Symposium on Computer Architecture (ISCA)*, 2021, pp. 1083–1095.

[136] Y. Wang, J. Lin, and Z. Wang, "An energy-efficient architecture for binary weight convolutional neural networks," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 26, no. 2, pp. 280–293, 2017.

[137] T. Wolf, L. Debut, V. Sanh, J. Chaumond, C. Delangue, A. Moi, P. Cistac, T. Rault, R. Louf, M. Funtowicz, J. Davison, S. Shleifer, P. v. Platen, C. Ma, Y. Jernite, J. Plu, C. Xu, T. L. Scao, S. Gugger, M. Drame, Q. Lhoest, and A. Rush, "Transformers: State-of-the-art natural language processing," in *Proceedings of the Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, 2020, pp. 38–45.

[138] Y. N. Wu, P.-A. Tsai, S. Muralidharan, A. Parashar, V. Sze, and J. Emer, "HighLight: Efficient and flexible DNN acceleration with hierarchical structured sparsity," in *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture*, 2023, pp. 1106–1120.

[139] Y. N. Wu, P.-A. Tsai, A. Parashar, V. Sze, and J. S. Emer, "Sparseloop: An analytical approach to sparse tensor accelerator modeling," in *Proceedings of the 55th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2022, pp. 1377–1395.

[140] G. Xiao, J. Lin, M. Seznec, H. Wu, J. Demouth, and S. Han, "Smoothquant: Accurate and efficient post-training quantization for large language models," in *International Conference on Machine Learning*. PMLR, 2023, pp. 38 087–38 099.

[141] G. Xiao, Y. Tian, B. Chen, S. Han, and M. Lewis, "Efficient streaming language models with attention sinks," *arXiv preprint arXiv:2309.17453*, 2023.

[142] D. Yang, A. Ghasemazar, X. Ren, M. Golub, G. Lemieux, and M. Lis, "Procrustes: A dataflow and accelerator for sparse deep neural network training," in *Proceedings of the 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2020, pp. 711–724.

[143] J. Yang, Z. Zhang, Z. Liu, J. Zhou, L. Liu, S. Wei, and S. Yin, "FuseKNA: Fused kernel convolution based accelerator for deep neural networks," in *Proceddings of the IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, 2021, pp. 894–907.

[144] S. Yang, Y. Sheng, J. E. Gonzalez, I. Stoica, and L. Zheng, "Post-training sparse attention with double sparsity," *arXiv preprint arXiv:2408.07092*, 2024.

Authorized licensed use limited to: University of Electronic Science and Tech of China. Downloaded on October 31,2026 at 10:33:52 UTC from IEEE Xplore. Restrictions apply.

[145] T. Yang, F. Ma, X. Li, F. Liu, Y. Zhao, Z. He, and L. Jiang, "DTA-Trans: Leveraging dynamic token-based quantization with accuracy compensation mechanism for efficient Transformer architecture," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 42, no. 2, pp. 509–520, 2022.

[146] Y. Yang, J. S. Emer, and D. Sanchez, "ISOSceles: Accelerating sparse CNNs through inter-layer pipelining," in *2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2023, pp. 598–610.

[147] A. Yazdanbakhsh, A. Moradifirouzabadi, Z. Li, and M. Kang, "Sparse attention acceleration with synergistic in-memory pruning and on-chip recomputation," in *Proceedings of the 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2022, pp. 744–762.

[148] H. You, Z. Sun, H. Shi, Z. Yu, Y. Zhao, Y. Zhang, C. Li, B. Li, and Y. Lin, "ViTCoD: Vision Transformer acceleration via dedicated algorithm and accelerator co-design," in *Proceedings of the IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, 2023, pp. 273–286.

[149] M. Zaheer, G. Guruganesh, K. A. Dubey, J. Ainslie, C. Alberti, S. Ontanon, P. Pham, A. Ravula, Q. Wang, L. Yang, and A. Ahmed, "Big bird: Transformers for longer sequences," *Advances in neural information processing systems*, vol. 33, pp. 17 283–17 297, 2020.

[150] S. Zhang, S. Roller, N. Goyal, M. Artetxe, M. Chen, S. Chen, C. Dewan, M. Diab, X. Li, X. V. Lin, T. Mihaylov, M. Ott, S. Shleifer, K. Shuster, D. Simig, P. Singh Koura, A. Sridhar, T. Wang, and L. Zettlemoyer, "OPT: Open pre-trained transformer language models," *arXiv preprint arXiv:2205.01068*, 2022.

[151] J. Zhao, P. Zeng, G. Shen, Q. Chen, and M. Guo, "Hardware-software co-design enabling static and dynamic sparse attention mechanisms," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 2024.

[152] Y. Zhao, C.-Y. Lin, K. Zhu, Z. Ye, L. Chen, S. Zheng, L. Ceze, A. Krishnamurthy, T. Chen, and B. Kasikci, "Atom: Low-bit quantization for efficient and accurate LLM serving," *Proceedings of Machine Learning and Systems*, vol. 6, pp. 196–209, 2024.

[153] Z. Zhou, J. Liu, Z. Gu, and G. Sun, "Energon: Toward efficient acceleration of Transformers using dynamic sparse attention," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 42, no. 1, pp. 136–149, 2022.

Authorized licensed use limited to: University of Electronic Science and Tech of China. Downloaded on October 31,2026 at 10:33:52 UTC from IEEE Xplore. Restrictions apply.